
RoLA: SCALING LINEAR ATTENTION’S RECALL WITH SEQUENCE LENGTH

A PREPRINT

Blake Bottum

Independent Researcher

blakebottum.research@gmail.com

ABSTRACT

Softmax attention stores a key and a value for every token, so its per-token compute and stored state both grow with L , the context length. Linear attention bounds both by folding the history into a fixed-size recurrent state, and gives up associative recall: every token writes into that state, and a query reads their superposition.

That state is a single value slot every token writes into, which suggests many slots with tokens routed among them, and recent architectures build versions of this. Routing, wherever it appears, is a producer of query and key features, so each such design is the recurrence it already ran under a new feature map. Which recurrence to route is a choice, and we take linear attention, the class softmax attention itself belongs to. When its write coefficients depend on the input rather than the stored state, a routed memory is that class at feature dimension N , its write routing the key features and its read routing the query.

RoLA (Routed Linear Attention) is the general form this licenses: a factorized hierarchy of per-level entmax distributions, reaching $\prod_l b_l$ features from $\sum_l b_l$ router outputs. Its features decay by write weight received and not by elapsed time, and its kernel bills the tiles a token touches. That kernel addresses $N = 65,536$ features, and it meets softmax attention’s per-token arithmetic at $N = L$, the size at which a comparison between the two is capacity-fair. [\[PLACEHOLDER: Headline result, with its controls, on rental-class hardware under the ratified protocol.\]](#)

Keywords linear attention · recurrent memory · sparse routing · entmax · associative recall · efficient transformers

1 Introduction

Softmax attention retains every token it has read, a key and a value apiece. It scores each query against all of them, so compute per token and stored state both grow with the context [\[Vaswani et al., 2017\]](#).

Linear attention bounds both. A feature map that factors the query-key similarity folds the history into a recurrent state of fixed size, so a read is one contraction and nothing accumulates with length [\[Katharopoulos et al., 2020\]](#). What it gives up is associative recall. The state is one destination for every write, so the pairs deposited in it lie on top of one another. A query recovers a value only while nothing else has landed where it was stored. Recall explains 82 percent of the perplexity gap between attention and the gated-convolution models where that gap has been decomposed [\[Arora et al., 2024a\]](#).

The single destination is the thing to change. That state is one value slot, so provide many and route tokens among them: a token deposits where its content sends it, and a query reads there.

A small instance fixes what that buys. Take a head with three features and a vocabulary of five items, where any one sequence names two of the five and a later query asks for one of the two. If the two live items land in different features, each read returns what was stored. If they land in the same feature, the read returns a blend of the two and nothing downstream can separate them.

What the head has to provision for is therefore the number of items a read must keep apart at once. That is two here, and never the five the vocabulary holds or the length of the sequence that named them. Section 5 turns that count into a graph and the head’s size into its chromatic number.

Several architectures of the past year build versions of this, differing in how a token selects slots and how a slot forgets [Afzal et al., 2026, Xiao et al., 2026, Du et al., 2025, Pan et al., 2025, Cabannes et al., 2026]. Routing is a technique for building keys, a producer of the query and key features that any architecture carrying queries, keys and values needs. A routed design is therefore the recurrence it already ran with a new feature map in front of it.

That reading covers the whole family, and several members place themselves inside linear attention already, each stating it for the one router it builds [Pan et al., 2025] (Appendix A.15).

What is left to choose is the recurrence, and the literature has routed both linear attention and the delta rule. We take linear attention, for four reasons. It is the class softmax attention itself belongs to: attention’s cache appends a key and a value per token and modifies nothing already stored. At $N = L$ it is a member like any other.

What identifies the class is that folding a chunk’s transitions into its keys asks them to commute across tokens, so the order they are applied in cannot matter. The delta rule’s transitions fail that [Schlag et al., 2021] (Section 3). What we are trying to match is softmax attention, which the class contains, so the delta rule’s extra machinery buys nothing here.

Working affine is working inside the same species as the architecture to be matched, so parity at matched capacity is a hypothesis. This paper is its test. The capacity theory sizes the state a task demands, the cost arithmetic prices the addressing (Section 6), and Section 7 states the protocol. The rest of the regime arrives with the same affineness. A readout’s exactness survives however far a write spread, the backward pass reconstructs no state history, and a per-token bill follows the features a token touches.

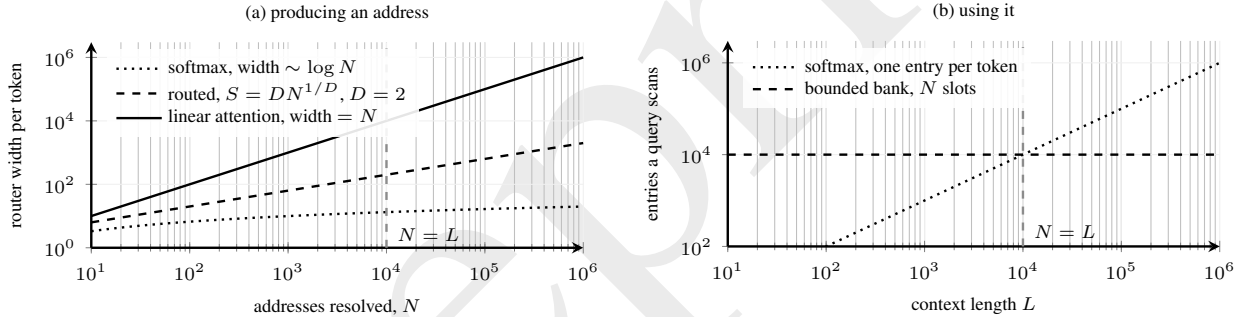


Figure 1: **What an address costs, on both sides.** The abscissa in (a) is the number of slots a design resolves and the ordinate is the router width a token produces to reach them, which is the query–key width for a similarity or a dense feature map and $S = \sum_l b_l$ for a factorized router. Softmax attention’s similarity multiplies evidence across dimensions, so $\Theta(\log N)$ features separate N addresses. Linear attention’s is a sum over features, so its capacity is the feature dimension itself. A factorized router lies between, at $DN^{1/D}$ for fixed depth and logarithmic once depth grows with capacity. Attention is the cheaper producer, and (b) is what it pays for that: its addresses are the tokens themselves, so the entries a query scans grow with the context, where N slots hold whatever the context is. The two meet at $N = L$, where a comparison is capacity-fair (Section 5.3). The curves are structural laws with constants suppressed.

Take a memory whose slots update independently and whose readout is linear, with write and read coefficients that are functions of the input alone. That recurrence is linear attention at feature dimension N , its two routing vectors in the roles of key and query features (Section 3). The statement holds for any router, so it describes the general form of a routed linear-attention head.

RoLA is the head this licenses, and its feature map is factorized. It has D routing levels, level l offering a choice among b_l branches, with one sparse distribution per level. A leaf is a complete address, one branch chosen at every level, and it names one slot.

Capacity is then the product of the widths where the router’s output is their sum, $\prod_l b_l$ slots from $\sum_l b_l$ numbers, which is the mixed-radix arithmetic the design runs on.

The contributions follow that order.

1. The reduction, and the recent routed family located in it, including the update rules that fall outside it (Section 3).

2. The capacity theory that sizes N against what a retrieval contract demands, with the rule that a comparison matches addressing entropy, the log of the number of addresses a design resolves, in place of parameter count (Section 5). Addressing entropy is, in short, how many distinct places a design can tell apart, counted in bits.
3. The hierarchical entmax feature map, in the three wirings of its read and write supports. Its decay runs on a clock of received write mass, learned per level and held off the gradient, which is what removes the state history from the backward pass (Section 4).
4. The kernel that bills the slots a token touches and not the slots it owns, with the cost model that plans it. It runs at $N = 65,536$ and meets attention’s arithmetic at $N = L$ (Section 6).

The released layer takes the levels, their widths, each level’s activation and side assignment, and the clock as configuration (Appendix A.15).

Language modelling at any scale is the sequel’s subject.

2 Bounded-state memories

Softmax attention [Vaswani et al., 2017] and linear attention are two instances of one attention form, an output built by contracting a query against per-token entries whose keys are fixed when written. The kernel reading that groups the two is Tsai et al. [2019]’s, and the condition on keys is ours. The name is also used loosely, as an umbrella for the linear-time recurrent family at large, the delta rule and its gated variants included [Yang and Zhang, 2024]. We use it in the strict sense of Katharopoulos et al. [2020] throughout, under which the delta rule and its variants fall outside the form, their updates reading as fast weights [Schlag et al., 2021]. Those authors show the two readings to be formally equivalent, so the boundary is one of scope alone.

Linear attention removes attention’s pairwise score. A feature map ϕ applied to queries and keys alike writes the similarity as an inner product $\phi(q_t)^\top \phi(k_j)$, so the sum over earlier tokens folds into a running state $M_t = \sum_{j \leq t} \phi(k_j) v_j^\top$ [Katharopoulos et al., 2020]. That state is a matrix of fixed shape, set by the feature dimension d_{qk} and the width of a value. A read contracts it once against $\phi(q_t)$. Per-token compute and stored state then stop growing with the context.

The feature dimension is also what bounds recall. A stored value survives retrieval only while the feature vector that addressed it stays separable from the rest, so d_{qk} and not the token count sets how many key-value pairs the state holds at once. Multi-query associative recall makes the gap measurable [Arora et al., 2024a]. A sequence presents many key-value pairs and then queries a subset of the keys. Accuracy falls as the pair count approaches what the feature dimension can separate, where softmax attention holds accuracy by keeping a separate entry per token.

Much of the line since has varied the update rule, and that axis reads as a ledger of what a write may depend on (Appendix A.15).

3 Routed slot memories as linear attention

This section establishes the identification the rest of the paper runs on: a routed slot memory is linear attention under a particular feature map. Recent bounded-state designs replace linear attention’s single recurrent state with a bank of N slots [Peng et al., 2022, Zhang et al., 2024b, Afzal et al., 2026, Xiao et al., 2026]. A router decides which slots each token writes and which its query reads.

Read as mechanisms they look like anything but feature maps. The recurrence is indifferent to what produced the vector: a routed slot memory is linear attention under a particular feature map, and the proposition names it.

Proposition 1 (Routed slot memories are linear attention). *Consider a memory of N slots, slot s holding $M_{t,s} \in \mathbb{R}^{d_v}$, driven at token t by a write vector $w_t \in \mathbb{R}^N$, a read vector r_t , a value $v_t \in \mathbb{R}^{d_v}$ and a retention vector a_t , with $M_{t,s} = a_{t,s} M_{t-1,s} + w_{t,s} v_t$ and $y_t = \sum_s r_{t,s} M_{t,s}$. Assume slot separability (no cross-slot mixing), input-affine writes (w_t, r_t, v_t and a_t are functions of the input stream and not of M), and a linear readout. Then with $a_{t,s} = 1$ the unrolled recurrence is linear attention at feature dimension $d_{qk} = N$ under $\phi(k_t) = w_t$, $\phi(q_t) = r_t$, $M_t = \sum_{j \leq t} \phi(k_j) v_j^\top$; input-dependent $a_{t,s} \in (0, 1]$ gives gated linear attention, a_t acting as a per-feature decay on that map.*

Separability makes the update N independent scalar recurrences. Affine writes make every coefficient in them a function of the inputs alone, so the sum is a feature map evaluated on the stream (Appendix A.15).

Reading a routed memory that way costs nothing and fixes the vocabulary. Table 1 is the dictionary. It is why the sections after this one speak linear attention in place of routing. What a design chooses is a feature map, and the routing words name properties of that map.

Table 1: **The reduction as a dictionary.** Proposition 1 carries the left column onto the right one, each entry an identity rather than an analogy, so what the linear-attention literature already asks of a feature map is what to ask of a router.

Routed slot memory	Linear attention
write routing w_t	key features $\phi(k_t)$
read routing r_t	query features $\phi(q_t)$
one of the N slots	one feature dimension
the bank, and its contents	the state M_t , at $d_{qk} = N$
retention $a_{t,s}$	per-feature gating
a level’s entmax allocation	a factor of a sparse feature map, with true zeros
a leaf address	the index of a feature

The normalized regime is a fact about RoLA and not a simplification of it, since write mass, the clock reading it and entmax as a projection onto the simplex all need a magnitude only a nonnegative coefficient supplies. Keeping it is what puts the head on softmax attention’s ground, and it scopes the paper (Appendix A.15).

The proposition’s condition is a condition on the transition, and the field reads as a ladder through it. Every modern fixed-state architecture writes its state S_t as a linear-in-state recurrence $S_t = A_t S_{t-1} + b_t$, with the transition A_t and the input term b_t functions of the input stream alone. What separates the classes is the structure of A_t .

The identity is undecayed linear attention. Diagonal transitions are what elementwise decay and gating produce, so an unrolled product of them collapses to a per-token elementwise reweighting the keys absorb. A rank-one modification of the identity fails to commute, and dense A_t is the general linear RNN; Appendix A.15 walks the ladder.

Table 2 collects the family across that ladder, the designs on the diagonal together with the routed ones built on the other recurrence. Their routing is key production like any other, so the reading above covers them all unchanged. What differs is the rule the router feeds, a transition off the diagonal for the delta-rule members and a state-dependent write for the gradient-descent one. The factorization the attention form runs on is unavailable around those.

Table 2: **The routed family as feature maps.** Every row is one recurrence under a produced feature map, its routing key production like any other, so the feature dimension N compares across rows; for the diagonal rows, ours included, Proposition 1 writes that recurrence in the attention form. We read N off the largest configuration each paper reports through that paper’s own address formula, 4^5 from RAM-Net’s d_p^U at $d_p = 4$, $U = 5$, Nc from SSE’s N partitions over a base row count c , and $(d/4H)^2$ from SDM’s formula at its 8B width. Where a paper states no factor we leave it symbolic. “w/r” is the sparsity of the write and read supports, sp. for sparse. The recurrence column types each row’s transition A_t with the clock or the rule that fills it, and the diagonal rows are the class this paper works in, the choice Section 1 states. In every row but ours the sparsity pattern is a fixed property of the design, where RoLA sets it per level: the default is sparse-write/dense-read, and the level split of Section 7, one level sparse on the write and another sparse on the read, realizes sparse/sparse. Sparse Delta Memory, abbreviated SDM in the first column, is unrelated to Kanerva’s sparse distributed memory [Kanerva, 1988]. FwPKM addresses a product-key bank of 512^2 slots, and MoM runs four memories and a shared one, classified here as the instantiation it evaluates rather than as its pluggable framework, whose plain-additive form (its Eq. 8) is diagonal.

Method	What ϕ is	N	w/r	Recurrence
Raven [Afzal et al., 2026]	sigmoid, top- k	512	sp./dense	diagonal, decay on selection
RAM-Net [Xiao et al., 2026]	Kronecker softmax	4^5	sp./sp.	diagonal, write mass at one global rate
SSE [Pan et al., 2025]	top- k , partitions	Nc	sp./dense	diagonal, timestep gate
Mamba-2 [Dao and Gu, 2024]	dense, input-dependent	256	dense/dense	diagonal, timestep gate
SDM [Cabannes et al., 2026]	top- k scores	2.3×10^5	sp./sp.	$I - \beta_t k_t k_t^T$, the delta rule
FwPKM [Zhao and Jones, 2026]	product-key top- k	2.6×10^5	sp./sp.	state-dependent, a local gradient step
MoM [Du et al., 2025]	softmax, top- k	$5 d_{qk}$	sp./sp.	$I - \beta_t k_t k_t^T$, gated delta as evaluated
RoLA (ours)	mixed-radix entmax	[PLACEHOLDER: largest trained N]	per level	diagonal, per-level write-mass clock, detached

Raven, SSE and RoLA write sharply while reading densely, and they pay differently for it (Appendix A.15).

The recurrence off the diagonal gives up the chunked parallel form, which is one of the reasons Section 1 gives for taking the affine side. Whether the affine side is itself the limitation is the hypothesis this paper tests (Appendix A.15).

The proposition places every row of the table, ours included, since routing is a way of building a feature map and routing per se is novel nowhere. The form holds for any router, so a routed linear-attention head is a setting of it and not a design beside it. What a proposal still owes is its map, its clock and its kernel.

Three things separate ours from the rest of the family. The first is scale. Our map is exact entmax over a factorization sized for N between 10^5 and 10^6 , three orders above the 10^2 to 10^3 at which the routed alternatives report a trainability fix (Appendix A.15). This paper’s kernel addresses $N = 65,536$ (Section 6), and Table 2’s open cell is the largest trained one.

The second is the decay. Advancing by received write mass is prior art we carve out below; ours are two properties of that clock, a rate learned per level and exact detachment in place of a proxy gradient (Section 4.4).

The third is a kernel at $N = L$ (Section 6). Without one, describing a slot memory as linear attention at $d_{qk} = 10^6$ restates the design and does not price it (Appendix A.15).

RAM-Net is the closest prior work and anticipates much of the mechanism, so we state the overlap in full [Xiao et al., 2026]. It partitions the input into U sub-vectors, softmaxes each and synthesizes a slot address as their Kronecker product, giving d_p^U slots, then truncates to the top K of them. Its update is affine and key-free, its decay a continuous power of the write weight a slot receives, and it ships sparse kernels that decode addresses without enumerating the product. Factorized product addressing, key-free affine slot writes, a normalized readout over addressed slots and write-driven forgetting belong to that work, and we claim none of them.

What we add is a hierarchy in place of a flat factorization, and exact entmax where RAM-Net truncates a softmax to its top K . Our rate is learned per level and detached, where its clock is one global exponent trained through a proxy gradient. Appendix A.15 states the overlap and the four differences in full.

4 RoLA

This section fixes the head the reduction licenses. A RoLA head keeps a state of N features, and its feature map gives each token key features saying which it writes and query features saying which it reads. Four parts are configurable, and the subsections below fix each: the level configuration, the activation each allocation comes from, the decay semantics, and the readout normalization. Allocations stay normalized and nonnegative, the regime scoped in Section 3.

4.1 Topology of the feature map

The map is a hierarchy of levels whose complete addresses are its features. Fix one batch element and one head. The map has D levels, level l of branch width b_l . A leaf address picks one branch per level, and the feature count is the product of the widths:

$$s = (d_0, \dots, d_{D-1}), \quad d_l \in \{1, \dots, b_l\}, \quad N = \prod_{l=0}^{D-1} b_l, \quad (1)$$

one feature per leaf. We write $d_l(s)$ for the branch leaf s takes at level l , and $S = \sum_l b_l$ for the total branch width. The sum is what a side emits, the product is what it addresses, and the gap is what depth buys. The levels are scales of the address, where log-linear attention makes its hierarchy over the sequence axis [Guo et al., 2025a]. A coarse level here selects a region of one fixed state, and every token passes through every level.

At token t each side emits one allocation per level, a distribution over that level’s branches saying where the side points:

$$p_{l,t,d}^r \geq 0, \quad \sum_d p_{l,t,d}^r = 1, \quad p_{l,t,d}^w \geq 0, \quad \sum_d p_{l,t,d}^w = 1, \quad (2)$$

with p^r the read allocation and p^w the write allocation. Each level and side carries its own activation. Softmax is dense, while the α -entmax family returns true zeros [Peters et al., 2019, Martins and Astudillo, 2016], meaning that a branch left out gets a true zero and not merely a small number. We use its order-1.5 member and sparsemax at order 2.

The support of such a solve is the set of branches it puts mass on, in other words the branches the side actually touches. Everything else gets a true zero. That support has a boundary moving continuously with the logits, so the task gradient recruits and releases branches. We write entmax for whichever order a level carries, and reserve α for the provisioning ratio of Section 5.

Each level carries one of two configurations, independent or tied, and a tied level is what the shared-support solve of Section 4.8 builds on (Appendix A.11).

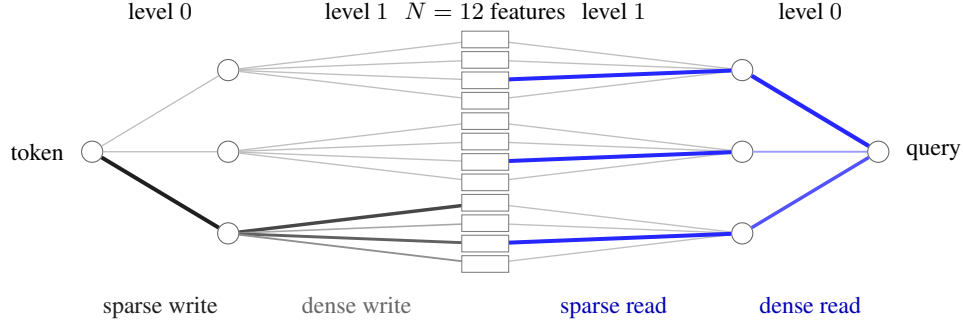


Figure 2: **The RoLA head.** Widths $b_0 = 3$ and $b_1 = 4$ give $N = 12$ features, one per leaf address, and that column is the state. The token enters at the left and its query at the right, each passing through the same levels, so level 1’s pattern repeats under every level-0 branch. A support drawn bold is sparse and one drawn as graded tint dense, black for the write side and blue for the read, each level assigning the two sides independently. The default configuration is sparse writes and dense reads at every level; drawn here is the sparsity split by level, sparse on the write at level 0 and on the read at level 1, which Section 7 measures.

4.2 Allocation and mass

A write into a leaf carries three jobs, and normalizing the allocation keeps them apart. The allocation says where, one positive scalar per side per token, g_t^r and g_t^w , says how much, and the value says what. The gain is the one of the three that reaches the mass alone, which is what makes it identifiable.

Halving the write gain and doubling the value leaves the numerator of the readout below where it was and halves the denominator. The item’s weight against everything else the state holds moves, and the output moves with it. Proposition 2 carries g_t^w explicitly for that reason. The three jobs give leaf factors

$$R_{t,s} = g_t^r \prod_l p_{l,t,d_l(s)}^r, \quad W_{t,s} = g_t^w \prod_l p_{l,t,d_l(s)}^w, \quad (3)$$

with $R_{t,s}$ and $W_{t,s}$ the weights leaf s receives in the read and in the write. Both gains are token-conditioned.

Each side carries one mass and no more, per-level masses collapsing into a side-wide product (Appendix A.11).

4.3 The value recurrence

Each leaf owns one value vector $M_{t,s} \in \mathbb{R}^{d_v}$, with d_v the value width. A token deposits its projected value $v_t \in \mathbb{R}^{d_v}$ into every leaf its write factor reaches. Earlier contents survive by a factor $\text{keep}_{t,s} \in (0, 1]$ fixed in Section 4.4, and the read contracts the state:

$$M_{t,s} = \text{keep}_{t,s} M_{t-1,s} + W_{t,s} v_t, \quad \text{num}_t = \sum_s R_{t,s} M_{t,s}. \quad (4)$$

The deposit is additive with input-dependent coefficients, so the recurrence is affine in the state, as Proposition 1 asks.

Each leaf also carries a scalar mass $z_{t,s}$ under the same recurrence, and the readout divides the two contractions:

$$z_{t,s} = \text{keep}_{t,s} z_{t-1,s} + W_{t,s}, \quad \text{den}_t = \sum_s R_{t,s} z_{t,s}, \quad y_t = \frac{\text{num}_t}{\text{den}_t + \eta}, \quad (5)$$

with $\eta > 0$ a small constant. The mass is an appended constant-one value channel, and the read gain cancels between numerator and denominator, so we fix $g_t^r = 1$.

The normalized readout has a consequence the rest of the paper leans on. A feature never written contributes zero to num_t and zero to den_t . Reading it returns nothing rather than noise, so the write support need not contain the read support for correctness, which is what makes the sparse-write, dense-read topology safe.

4.4 Decay on a write-mass clock

With decay disabled, $\text{keep}_{t,s} = 1$ and Eq. (4) is additive. The learned form ages a feature by how much it has taken in, not by how long it has existed. In effect, a feature that nothing writes to keeps what it holds indefinitely. Each level

carries a per-branch dial $\delta_{l,d} \in [0, 1)$, and a leaf’s rate is the product of the dials on its own address. The exponent is the write mass that leaf receives at this token, held off the gradient by a stop-gradient $\text{sg}[\cdot]$:

$$w_{t,s}^{\text{leaf}} = \prod_l p_{l,t,d_l(s)}^w, \quad \text{rate}_s = \prod_l \delta_{l,d_l(s)}, \quad \text{keep}_{t,s} = (1 - \text{rate}_s)^{\text{sg}[w_{t,s}^{\text{leaf}}]}. \quad (6)$$

The write gain g_t^w scales the deposit but stays out of the clock, which keeps the exponent in $[0, 1]$.

The clock ignores that gain by design, since a state-dependent share of the accumulated mass would cross the input-affine boundary of Section 3 (Appendix A.12).

In the gated linear-attention lineage the clock advances once per timestep, so a feature ages even when it takes no write [Yang et al., 2024b, Gu and Dao, 2023, Yang et al., 2024a, Qin et al., 2024]. Here an unwritten feature keeps every bit, which is the law we want once Section 5 sizes the state so that almost every feature is idle at almost every step (Appendix A.12).

Forgetting in proportion to writing rather than to elapsed time is prior art and we do not claim it, RAM-Net’s clock being the closest and Section 3 the carve-out. Raven gates decay on selection [Afzal et al., 2026], and usage-driven reallocation goes back to the differentiable-memory line [Graves et al., 2016, Rae et al., 2016]. Two properties of Eq. (6) are ours.

The model learns the rate per level and composes it across the hierarchy. A leaf’s rate comes from its own address, at S parameters, so a coarse region can stay durable while the features beneath it turn over.

The clock also runs detached. The stop-gradient leaves the recurrence affine in the state, so the backward pass needs no state history and no chunk-boundary snapshots (Appendix A), a claim we scope to the chunked gated-linear-attention family (Appendix A.12).

Inside that family we know of no other gate that is an algebraic function of the feature’s own write mass, the recent finest-grained forms sharpening the separation of gate from write allocation rather than tying them (Appendix A.12).

4.5 Structural requirements

A head is one mechanism, one bank of leaves under one readout, and a wiring is a choice of how the two sides address it. Gradient reaches the write producer and the value only through the leaves the read gathers. An empty intersection at a step severs that path rather than denting the accuracy. What a wiring owes is therefore a structural intersection and not a probable one, on every step and for every pair of logits. The mechanism under it is entmax’s support-restricted Jacobian, which gives an out-of-support logit zero gradient.

Disjoint supports are then permanent dead directions with no re-entry signal, the gap the top- k lineage patches by recruiting with injected noise [Shazeer et al., 2017]; Appendix A.11 works the mechanism through.

Correctness survives the cut. Untied sparse supports are exact already, the unwritten leaves of Section 4.3 being inert in both sums, and what the intersection buys is gradient rather than exactness (Appendix A.11).

Exactness then classifies what survives that cut, under either of two conditions per level. Under agreement both sides pin the level’s digit, at one touch a side, and the cue must carry that digit (Section 5). Under coverage one side spreads over all b_l digits and asks nothing of the cue there, at b_l touches on that side.

Coverage is what a guarantee for arbitrary logits needs: a read support of size r_l and a write support of size w_l are guaranteed to meet only if $r_l + w_l > b_l$. Concretely, two sets drawn from b_l branches can miss each other unless their sizes together exceed b_l .

Touches then price a guarantee, and the arithmetic fixes its width before any measurement. It is the all-coverage wirings the equation below governs, every level supplied dense by one side and none riding on the cue, which is what holds however underdetermined the cue. Such a wiring guarantees a read reaching $r = \prod_l r_l$ leaves and a write reaching $w = \prod_l w_l$, and those two widths multiply to the feature count.

$$r w = \prod_l b_l = N. \quad (7)$$

That is Eq. (1) read on the supports in place of the widths, so structural coupling with each side solving its own support is conserved work along a hyperbola. It scopes to those wirings and is not a requirement of exact addressing.

A contract whose cue determines the address at every level runs at $r = w = 1$ and still reaches all N features, and $r w < N$ says some levels ride on agreement in place of coverage, which is where the shared solve sits by design. The

two counts are structural envelopes and not per-token bills: entmax support is adaptive, a token’s realized support sits at or below the envelope, and a confident cue collapses toward the point case. The kernel bills the touches a token realizes (Section 6).

A guarantee holding for every pair of logits means at least one side’s realized support departs from an independent point mass. Table 3 is what that leaves, and the subsections below construct the three wirings that pay; Appendix A.11 derives the family as the guarantee’s solution set.

Table 3: **The wiring family.** Each row buys the intersection of Section 4.5 differently, and the subsections below construct the three that buy it at all. The envelope is the guarantee’s width rather than a token’s bill, and Eq. (7) governs the first two rows, which supply every level dense on one side. The shared support sits below the hyperbola by design, at both sides sparse. The last row is the wiring the law excludes, cheapest on both sides and structurally unable to promise an intersection at any step.

Wiring	Pays with	Sides per level	Envelope (r, w)	Touches	Exposure
Sparse write, dense read	touches	write pins, read dense, at every level	$(N, 1)$	$O(N)$	none on either side
Sparsity split by level	structure	each level dense on one side, sparse on the other	$(\sqrt{N}, \sqrt{N})$	$O(\sqrt{N})$	a block write, leaking mass into neighbors
Shared support	coupling	one support serving both sides	$rw \ll N$	$O(1)$	a cue diverging from the content, priced by ε
Untied sparse (excluded)	nothing	both sides sparse, independently solved	$(1, 1)$	$O(1)$	no guaranteed intersection

4.6 Dense reads

A dense read spreads over every branch of every level while the write stays a point mass, so the read support is the whole state and the write is one leaf. It is the configuration Section 4.1 names the default, and the one Eq. (10) prices at $N = L$ (Appendix A.11).

What it pays is touches, at the envelope $(N, 1)$ of Table 3: the only sparsity a kernel can harvest is on the write side (Appendix A.11).

4.7 Sparsity split by level

Because a leaf factor is a product over levels, the two sides can take their sparsity at different levels, which is the split Table 3 places at $(\sqrt{N}, \sqrt{N})$ (Appendix A.11).

The payment lands on the write, which becomes a contiguous block of features and leaks mass into its neighbours (Appendix A.11).

4.8 The shared-support solve

A shared-support level solves once on the averaged logits and splits the two sides inside the resulting support, which makes the intersection an identity and sparsifies both sides at once:

$$p = \text{entmax}\left(\frac{1}{2}(z^r + z^w)\right), \quad \sigma = \text{sigmoid}(z^r - z^w), \quad (8)$$

with z^r and z^w the level’s read and write logits. The read allocation is $p\sigma$ without renormalization, and the write allocation is $p(1 - \sigma)$ renormalized over the support of p .

A uniform scale on the read side cancels in the ratio of Eq. (5), so leaving the read unnormalized costs nothing. The write allocation must be a distribution, because it deposits mass. The two sides stay untied in value through σ while sharing one support. Averaging, in place of summing, keeps the tied case an identity, since $\text{entmax}(2z)$ and $\text{entmax}(z)$ differ.

This is the deliberate exception at the agreement end, both sides sparse at $O(1)$ touches and $rw \ll N$, its exactness resting on the one map sending a cue where the content went rather than on coverage (Appendix A.11).

We write ε for the resulting task-level error, what a shared support costs: it measures how often a task reads outside where it writes, or the reverse, strongly enough for one support to get it wrong. Its two sources are decomposed in Appendix A and its exposure in Appendix A.11.

5 Capacity and addressing

Under Proposition 1 RoLA’s capacity is a feature dimension, and its addressing is the structure of its feature map. The proposition names it: level widths multiply into capacity, while the map emits only their sum (Section 4.1).

Proposition 2 (Kronecker-structured linear attention). *Unrolled with decay disabled, a RoLA head is linear attention at feature dimension $d_{qk} = N$ under $\phi(k_t) = g_t^w \otimes_l p_{l,t}^w$ and $\phi(q_t) = \otimes_l p_{l,t}^r$, each a tensor product of D normalized allocations of widths b_l . The write gain of Eq. (3) scales the deposit, and the read gain is fixed at $g_t^r = 1$ by the normalized readout of Eq. (5). Capacity is their product $N = \prod_l b_l$ and the map emits their sum $S = \sum_l b_l$ per side.*

That gap is what depth buys, and the map pays for it. The feature vector is a product of per-level marginals, so some patterns over leaves are unreachable while the state itself stays full rank (Appendix A.13).

Factorization costs nothing where writes want point masses, since a delta on a leaf is the product of the per-level digit deltas; it charges spread read patterns instead (Appendix A.13).

Attention provisions $\exp(\Theta(d_{qk}))$ near-orthogonal directions and materializes $O(L)$ of them per sequence as its cache, where RoLA provisions N combinatorially and commits the features written into. Both are logarithmic-class addressing for an exponential space, and Appendix A.13 places that reading against the Hopfield capacity line and the worst-case recall bounds.

The quantity that compares them is addressing entropy: $\log N = \sum_l \log b_l$ for the state, $c d_{qk}$ for a feature map of dimension d_{qk} , with c a packing constant set by the interference one tolerates. Section 7.1 measures c by fitting it. Comparisons match on that quantity, not on parameter count: equalizing parameters while leaving one side an exponentially larger address space measures the mismatch, not the architecture.

5.1 Vocabulary families and retrieval contracts

What a head must address is which distinguishable symbols can occur at each position, the family $V_1, \dots, V_L$, and every statement below quantifies over such a family. Token identity gives the shared family and positional encoding the disjoint one, so those are the content-associative and positional cases (Appendix A.14).

A retrieval contract is, in short, the list of questions a task will ask the memory. Each query names a target weighting over the values its sequence wrote, and nothing in it says where those values sit. The pair of maps is a solver, and placement is design freedom.

Under the normalized readout of Eq. (5) that solver has two error-free cases, a read confined to features holding one value each and a read spreading over features the sequence never wrote (Appendix A.13).

5.2 Sizing

Sizing starts from injectivity, which means that no two histories may end up as the same memory. The state loses nothing to the pigeonhole if and only if the map from sequence to written configuration is one-to-one. An injective addressing is available whenever the items a family can present at one position number at most N (Appendix A.14).

We size for point-write assignment, each item entering one feature. An assignment that spreads an item over several features answers to the nonnegative rank of the contract matrix in place of a coloring. Against the contracts this paper instruments that bound degenerates to the one point writes already give (Appendix A.7).

Above the injectivity floor the binding object is a graph. Its nodes are the items a family can present, one node per item and not one per occurrence, and an edge joins two items when some sequence the distribution realizes asks for both of them separately. An assignment is a coloring of that graph, proper, no edge with one color at both ends, when the write map is injective on every set of items live at once.

Every feature then holds at most one live value, which is the first exact case above. The least N that serves the contract is the chromatic number of that graph (Figure 3). That is, the head needs one feature for every item a read has to keep apart at the same time.

Every quantity this paper calls a demand is that number evaluated on a particular graph, and provisioning to it leaves every fulfillment pattern available (Appendix A.14).

The quantity is also computable with certificates on the families the instrument builds, so Section 7.1 stamps a solved demand on a cell before it trains.

Three regimes settle that number, and Table 4 is the three.

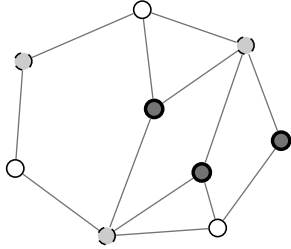
Table 4: **Three regimes and the chromatic number each carries.** The graph is the one the distribution realizes, and the least N that serves the contract is its chromatic number.

Regime	Graph it realizes	Chromatic number	What it asks of N
Random draws over a shared vocabulary	complete on $\min(L, V)$ items	$\min(L, V)$, a complete graph on k nodes taking k colors	$N \gtrsim \min(L, V)$ for exact recall
A disjoint family, one vocabulary a position	L -partite, so coloring by position is proper however dense the edges between positions grow	L	$N \gtrsim L$: vocabulary size multiplies the nodes and never the colors
A recency contract, reads never demanding a superseded item apart	interval overlap, and interval graphs are perfect	the window width, the most items live at one time	$N \gtrsim$ that width, below the count a sequence writes

Table 4’s three regimes are walked through in Appendix A.14: the first is what random draws over a shared vocabulary present, and the instrument of Section 7.1 runs in it, for that guarantee tier.

The third is induced by the contract, and its width is fixed before any architecture is chosen.

(a) a graph and a proper coloring



(b) a disjoint family

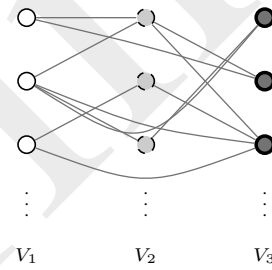


Figure 3: **Sizing as a coloring.** Nodes in (a) are the items a family can present, an edge joins two items some sequence asks for separately, and an assignment of items to features is a coloring, proper when no edge takes one color at both ends. The number of features the contract requires is the chromatic number, three here, and the triangle is what rules two out. Panel (b) is the disjoint family, one column per position: no edge lies inside a column, since symbols at one position never co-occur, so the columns grow as tall as the vocabularies V_i are large while coloring by position stays proper however dense the edges between columns become. A sequence realizes one item per position and asks for all of them, so it contributes a triangle across the columns and every item a sequence realizes carries edges into every other position; the drawn edges are four such sequences. The chromatic number is the column count, and that coloring is the one a factorized map expresses at S parameters, levels as position digits.

A contract bills two resources, and features are only the first: coloring the graph is a storage statement, and the width a cue must carry to select among the colors is the second (Appendix A.14).

Whether the maps can serve a contract is a relation between two functions on the family. Write addr^w and addr^r for the addresses the two sides assign. A contract sending a query symbol u to the stored symbol $\pi(u)$ is served when they agree:

$$\text{addr}^r(u) = \text{addr}^w(\pi(u)) \quad \text{for every } u, \quad (9)$$

Otherwise the read marginalizes densely over the digits it cannot fix, at an active-leaf count proportional to the matching instances, the bill attention pays for a multi-match query (Lemma 4). Attention satisfies Eq. (9) implicitly, $q \cdot k$ discriminating when the query carries what the key encoded.

The provisioning ratio $\alpha = N/L$ is the dial a design sets. It says, roughly, how many features the head carries per token of context. What it buys is room for a coloring, and a coloring that exists settles neither that the map can express it nor that training finds one. The map realizes only the colorings that factor into a linear entmax classification per level. That is the hypothesis the grid of Section 7.1 tests.

The number a contract requires is thus a functional of the distribution and the contract rather than a property of the head. What changes across families is how one comes to know it: a disjoint family asks for L and a designer knows that in advance, where a shared family asks for a chromatic number only measurement returns. This paper measures neither (Appendix A.14).

Sharing one feature between two symbols is safe when no retrieval map in the contract addresses one of them without the other. Two symbols that never co-occur may then share a feature for storage but not for retrieval, unless the cue the query addresses with covers everything that can land there (Appendix A.8).

None of this asks the map to be computed per sequence, layer 1’s family being the token vocabulary and depth supplying the dynamism thereafter (Appendix A.14).

How N must scale with L is a property of the sequences a task realizes rather than of L alone, and the general sizing question is one this paper leaves open (Appendix A.14).

Provisioning below the adversarial demand is a bet that the graph a design faces is smaller than the adversarial one, and Appendix A.14 gives the three things that shrink it.

5.3 $N = L$ as the adversarial contract

At $N = L$ the write side can be injective on instances, one color per occurrence, which covers the adversarial completion’s demand whatever graph a sequence realizes. Every feature then holds one value, the failure mode above is structurally unavailable, and residual error lies on the read side alone. Softmax attention hardcodes this regime: its addresses are free, one per occurrence and injective by position, so its key map learns what a key encodes and never where it lands.

That regime is an entry fee. Real sequences are redundant, so provisioning $N = \alpha L$ sizes the state to the information in a context rather than to its token count (Appendix A.14).

Setting $\alpha < 1$ is thus a choice of what may alias: provisioning below the chromatic number forces edges to take one color at both ends (Figure 3). Entmax triage is what chooses the cheap ones, with leaf mass decay metering the turnover of the features that stay hot (Section 4.4). Parity at $N = L$ is the worst-case guarantee, and α is set from the costly-edge subgraph a distribution presents, which this paper leaves unmeasured.

6 Cost

This section prices the head on the hardware that runs it. Accelerators issue work in blocks, so a sparse operator is billed for the blocks it touches and not for the entries it reads. Block-sparse and mixture-of-experts kernels design against that law [Chen et al., 2022, Guo et al., 2025b], and so do we.

What we add is where a routed head lands on it. Think of the state as a grid cut into blocks: a work item pairs an owner, that is, one block of B_C leaf features, with a tile of B_T tokens, and the device stages or skips it whole (Appendix A.16).

The quantization cuts both ways, and no average shows it: a candidate block holds fewer live features than the quantum it is billed for, and other candidate pairs hold nothing and skip every step. [PLACEHOLDER: At $N = 65,536$: mean live features per candidate block of 64, k-steps paid of four, and the share of candidate pairs that skip; at-scale cell, rental-class hardware, ratified protocol.] A cost model prices the quantum, not the mean.

Two consequences follow, and both invert the intuition that a sparse memory should touch as little as it can. Support size is nearly free once a support has paid for its tiles, so no cost argument remains for a hard per-token budget and Section 4.2 can let the activation set its own boundary (Appendix A.16).

The second consequence is that clustering, not sparsity, is the quantity worth optimizing: a tile is live if any of its B_T tokens touches it, so the kernel pays for the tokens’ combined footprint (Appendix A.16).

Nearby tokens choosing the same features is coherence, that is, tokens in one tile agreeing on where they write, and the factorized address supplies part of it by layout, since a coarse digit selects a contiguous region (Appendix A.16).

What that gives is structural, an address whose contiguity a flat address of the same capacity would have to learn. Hierarchy is a coherence prior in that sense, the cost-side counterpart of the addressing argument of Section 5 (Appendix A.16).

Provisioned capacity and resident memory come apart, which is the same argument on the storage side. The state is paged at a fixed granule and a page is admitted from the realized write support, so resident bytes scale with what a sequence wrote and not with the provisioned count (Appendix A.16).

In effect, a head may provision far more features than the memory it ever occupies. Translation costs a part in a thousand of the state traffic it rides on [PLACEHOLDER: translation overhead as a share of state traffic at the at-scale cell, rental-class hardware, ratified protocol].

Measurement recovers that law, separating a fixed per-tile term from a per-column one and pricing decay per k-step and not per tile, which Appendix B sets out with the cell that isolates each. Enabling the clock of Section 4.4 is therefore cheap in the regime it targets, since a leaf outside the write support adds no tiles.

6.1 The FLOP ledger

We price the comparison with softmax attention per pair and then count pairs, since attention’s per-token cost grows through the sequence and any one token’s price misstates the total. Attention pays $d_{qk} + d_v$ multiply-accumulates per query-key pair, which is $2d_v$ at the standard head shape $d_{qk} = d_v$ that the ledger below assumes, and RoLA pays about d_v per (token, leaf) pair (Appendix A.16).

The pair counts run the other way, causal attention visiting the triangle and RoLA the square its dense read sweeps. At $N = L$ the two prefill totals meet.

$$\text{attention} \approx 2d_v \cdot \frac{L^2}{2} = L^2 d_v, \quad \text{RoLA} \approx d_v \cdot LN = L^2 d_v. \quad (10)$$

The two factors of two cancel, so the totals are equal.

Two riders bound that equality. The square is the count for the wiring whose read is dense over the leaves. Where a level reads sparsely the tile skipping above takes the RoLA side below it, so Eq. (10) is an upper bound there and an equality here. Both sides also exclude input projections, and by different amounts. Fix $N = L = 65,536$ addressed as two levels of width 256, with $d_{\text{model}} = 1024$. The two excluded terms then read against the same $L^2 d_v$ as

$$\text{router} = 2L d_{\text{model}} S = \frac{1}{4} L^2 d_v, \quad \text{query, key, value} = 3L d_{\text{model}} d_{qk} = \frac{3}{64} L^2 d_v. \quad (11)$$

Attention’s projections are therefore three sixteenths of the router’s.

Decode returns the same answer. Over L steps the totals are $L^2 d_v$ against $LN d_v$, decode being the triangle paid one row at a time, and no phase tilts the ledger (Appendix A.16).

That size is the adversarial regime of Section 5.3, sparse writes with dense reads, so Eq. (10) says we reach attention’s arithmetic while paying to learn what attention gets free (Appendix A.16).

The affine recurrence makes one prediction about memory. Its backward pass reconstructs no history of the state, which Appendix A proves from the detached clock, so training memory per token is predicted flat in L , not measured [TODO: pending: the mode-axis runs, which wait on the native backward built after the forward freeze].

One term in the total stays linear in N whatever a wiring touches, since the kernel decides candidacy once per (owner, tile) pair, $NL/(B_C B_T)$ of those decisions over a prefill. Those are taken as word operations over the branch bitmaps the routing already publishes.

A bit of bookkeeping runs against a multiply-accumulate at a ratio near 10^{-4} , which we have not measured and state only to an order of magnitude. The slope-zero line of Figure 5(b) is therefore predicted to flatten onto a floor of that order and not onto zero.

We keep the term, and Appendix A.9 works out why removing it would cost more than it does, together with the structural caveat that the term scales with NL where the mechanism that would retire it stays fixed.

We claim parity at capacity-fair $N = L$, matched on addressing entropy, and not a throughput multiple at long L . Any fixed-state model wins that eventually and by construction. Concurrent work already reports large multiples over FlashAttention at 131K [Ghriss and Chakraborty, 2026], a number that measures the regime rather than the architecture.

Memory feed and exposed latency bind this kernel, not arithmetic. A model counting work alone cannot close that gap. We report the law, and we release the implementation. What the evidence behind the model covers, and where it stops, is Appendix B.

[TODO: Artifact URL once minted.]

7 Experiments

Two questions follow from the theory, and this section answers both: whether the capacity account predicts where retrieval fails, and what each wiring costs a token. The grid that answers the first is also this paper’s evidence that the head trains. Every number reported here is produced by the head of Section 4 on the semantics stated there, and where a measurement is still missing a placeholder names it, the repository’s MANIFEST.md listing the run that fills each one.

7.1 The capacity theory’s instrument

The first question is whether the capacity account predicts where retrieval fails. Section 5.2 makes that account a number, the demand, which is the chromatic number of the co-occurrence graph a distribution realizes and not the count of items a sequence writes. Multi-query associative recall leaves that account untested [Arora et al., 2024a], and the contract is what stops it (Appendix A.17).

A query re-presents the key its pair was stored under, so the read cue is the write content, and a head passes with one symmetric addressing map, never learning a correspondence between the two sides. That is also the regime in which shared-support routing holds Eq. (9) structurally (Section 4.5), so the wiring whose exactness most needs testing is the one the task exempts.

Presence recall is the repair, and it is multi-query associative recall with two repairs made to it: the cue is decoupled from the content by a frozen map, and the answer is one bit. We draw a map π once at random, taking every token of the vocabulary to a different token, $\pi(x) \neq x$, and freeze it for the life of a condition. A sequence is a string of tokens, and the supervised output at position t is one bit, whether $\pi(x_t)$ occurred anywhere before t . For example, where π sends *cat* to *dog*, a position holding *cat* answers yes if and only if *dog* came earlier in the sequence. The task sits in the synthetic-recall lineage that runs from associative retrieval [Ba et al., 2016] through induction heads [Olsson et al., 2022] to the multi-query grids and their state lower bound [Arora et al., 2024a,b, Jelassi et al., 2024]. Beside it stand the formal-language and long-context suites, which probe other axes [Delétang et al., 2023, Liu et al., 2023, Hsieh et al., 2024].

Each repair has a precedent, and neither precedent composes the two. A one-bit membership answer is what meta-learned Neural Bloom Filters return [Rae et al., 2019]. There the probe is the stored item itself, so nothing asks a head to learn a correspondence between two sides. Indirection is what Pointer Value Retrieval isolates [Zhang et al., 2021]. Its pointer is positional, supplied inside the instance under a rule the model is told, and its output is a value. Ours is a token-to-token bijection frozen across the dataset that the model has to learn in weights, and pointer chasing has since been studied for the depth it costs [Sanford et al., 2024]. A frozen dictionary learned in weights is MAD’s memorization task [Poli et al., 2024], where it stands alone and never meets an in-context query. Composing a weights-resident map with an in-context prefix question is what puts address agreement, Eq. (9), under measurement.

The cue and the content are different tokens by construction, so a head that answers has to store x where a later token pointing at x will look. That is, Eq. (9) becomes a property training has to find rather than one the cue supplies (Appendix A.17).

The answer being one bit is what leaves the feature count alone in the measurement, a multi-query lookup mixing addressing with retrieval where presence recall asks only whether the target is there (Appendix A.17).

The negatives supply separation from the other side, so the contract forces every pair of items a sequence realizes together onto distinct features. The demand is that graph’s chromatic number, and the collapse threshold keeps the form Section 5.2 gives it.

A grid that sweeps a quantity it cannot compute is sweeping a knob, so every segment solves the demand of the graph it actually realized and each cell sits at a demand that is solved and not assumed (Appendix A.17).

We run two conditions, complete graphs at demand 16 and at demand 64, at sequence lengths 64 and 256. Each sequence draws half its condition’s pool, so the demand and the count a sequence writes sit a factor of two apart in every cell.

We run four routed columns over the feature count N on each, the three wirings of Section 4.5 and a dense control at one level, with softmax attention the only other party and never on that axis (Appendix A.17).

The demand-16 ladder runs from the demand itself upward, so the crossing is measured at the higher demand and bounded at the lower. Either way the provisioning ratio α of Section 5.2 runs through one against a demand that was measured (Appendix A.17).

Every cell here is a trained model, its router and per-level clock optimized end to end and not fixed by construction. We run no head-to-head against the routed family of Table 2, because there is no size both sides run at (Appendix A.17).

We enter attention twice. It runs first at a key width ample for every demand, which is the instrument floor: it establishes that the cells are solvable at all, and no routed column can supply it (Appendix A.17).

It runs second as a second validation, on the axis its own capacity law lives on. A head of key width d_{qk} packs $\exp(c d_{qk})$ addresses at a packing constant c (Section 5), so its threshold moves with the logarithm of the demand where a routed one moves with the demand itself.

Sweeping d_{qk} over the same solved graphs makes c a measurement of ours, and we label c fitted wherever it appears. Three solved demands enter it, the middle one running attention alone, so the slope is a fit through three points (Appendix A.17).

We record what the account predicts before the runs, each prediction with what would falsify it.

1. A ratio, not a rung. Demand 64 needs four times the N that demand 16 needs, at the same shape of curve. Each condition’s threshold sits at its own solved demand rather than at the count of items a sequence draws, which sits a factor of two below it in every condition and which the ladders resolve. A saturation point that stays put as the solved demand moves falsifies it.
2. Attention’s own law. Its threshold in d_{qk} is linear in the base-two logarithm of the demand at positive slope $1/c$, over solved demands 16, 32 and 64. A threshold flat in that logarithm falsifies it, and so does one linear in the demand itself, which would collapse the two axes onto one.
3. The cue map is a storage cost for one wiring only. The covering wirings supply every level dense from one side and ask nothing of the cue there, so a divergent cue leaves their thresholds where the demand puts them and moves the training it takes to reach them. Shared-support routing produces one support per token for both sides, and under π that support has to carry the union $\{f(x_t), f(\pi(x_t))\}$, its own write address together with the address its target wrote at. Section 4.5 scopes that as canonicalization through the one map. We predict its threshold sits at or above the covering wirings’ at both demands, and that it is the column a longer schedule moves. A threshold equal to theirs at both demands falsifies it, and would say the exactness never rested on the cue carrying the content.

One more contrast runs at the lower demand, about the clock rather than the feature count. The write-mass clock of Section 4.4 advances a feature in proportion to the mass it takes in; a clock advancing every feature in the write’s support by a fixed step would tick them all fully. The two coincide on an exact point write, that is, wherever a token writes to one feature and no more, so the sparse-write column is a control cell and the clocks separate only where writes spread (Appendix A.17).

We record three predictions here as well, each with its falsifier (Appendix A.17). The write-mass clock sits at its decay-off twin on both spreading wirings, the fixed-step clock degrades with the width of the write support, and the point-write control pair holds. The fixed-step clock is an instrument for this contrast alone, and it runs on the training oracle, which is the engine every cell of the grid trains on and the only one that implements it.

Cells are small, since the axis is addressing and cheap cells buy breadth. Table 6 in Appendix A.10 is the configuration, held identical across every column.

We size the conditions, their ladders in N and the attention axes as 204 cells at the first stage, the rate sweep tripling every column, and the clock contrast adds 108 more. Every panel below is a placeholder until that tier runs.

[PLACEHOLDER: Accuracy against N for the three wirings and the dense control, one panel per demand, plotted against each cell’s solved demand, with the attention reference cells marking the instrument floor.]

[PLACEHOLDER: (a) Collapse threshold in N against solved demand for the four routed columns, and attention’s threshold in d_{qk} against $\log_2$ of the same demands, with c fitted.]

[PLACEHOLDER: (b) Latency against d_{qk} for softmax attention, with the RoLA latency points and the $N = L$ parity cell.]

Figure 4: **The two capacity axes, and what they cost.** Panel (a) is the exhibit both legs of the grid build: a routed threshold predicted linear in the demand, and attention’s predicted linear in its logarithm, over one set of graphs whose demands the solver certified. It is read on the falsification list above, and its legend labels c fitted. Panel (b) prices the same addressing, d_{qk} for a monolithic kernel and N at router width S for a routed one. Monolithic kernels climb with the $d_{qk}d_v$ they touch per token and stop where standard kernels no longer exist, the infeasible region marked rather than omitted. Attention is flat in d_{qk} but pinned at its $O(L^2)$ floor. What panel (b) is built to show is how little the RoLA curve climbs across that axis, measured to $N = 65,536$ and anchored at Eq. (10)’s parity point, with $N = 10^6$ scheduled as a further cell [TODO: pending: the $N = 10^6$ performance cell]. Points carry regime labels and paired timings, and no curve extends past them.

The same generator carries contracts and graph families this paper leaves unrun, each extending the instrument in one direction, and Appendix A.17 lists them. This paper’s confirmation runs on the one contract that isolates the feature count.

7.2 Cost surfaces for the three wirings

The second question is what read sparsification costs and what it buys. The kernel gains of Section 6 need a query to touch a small part of the state, where the default configuration reads densely. The three wirings run at identical N and L , so a difference between them is the wiring.

We sweep the wirings over three knobs the cost and the learning condition both depend on. They are the occupancy s , in other words how much of the memory a sequence ever touches, the coherence γ of Section 6, and the alignment a , the fraction of reads whose support meets the write that stored the target.

Section 4.5’s law places the three: each is one schedule for a structurally guaranteed read-write intersection, and the second column of Table 5 is what each gives up to hold it. The ablation measures which payment is cheapest on task quality, not which mechanism is better.

The touch exponents are the falsifiable part, since they are slopes and not constants. The envelope of Section 4.5 is a ceiling on what one token may touch, so adaptivity moves the constant a confident cue realizes and not the rate.

On log axes against N at fixed L , the three wirings predict slopes 1, $1/2$ and 0, and Figure 5(b) is that exhibit. Where the wirings actually cross in wall-clock time is a separate measurement on the same cells, the constants being tile quanta rather than touch counts (Section 6). We measure the crossovers.

A trained model enters the same figure as a marker, at its measured s , γ and a . That locates the cell on the knob axes and prices nothing, since a cost quoted for a cell is timed on the cell.

[PLACEHOLDER: (a) Quality surfaces for the three wirings at identical N and L over the occupancy, coherence and alignment axes, with the trained cells’ measured statistics marked.]

[PLACEHOLDER: (b) Touched features per token and measured latency against N at fixed L , log-log, one line per wiring.]

Figure 5: **The three wirings as surfaces.** Panel (a) sweeps the three knobs s , γ and a above, with N and L held identical across wirings, so a gap between surfaces is the wiring and nothing else; trained cells appear as markers at their measured (s, γ, a) , which locates them on the axes and prices them nowhere. Panel (b) is the falsifiable exhibit of Eq. (7): slopes 1 and $1/2$ for dense reads and the level split, with the shared support’s slope 0 placed relative to the hyperbola rather than on it, and the wall-clock crossovers measured rather than inferred, since the constants are the tile quanta of Table 7.

Table 5: **Read sparsification, three wirings** at matched addressing entropy against the dense-read reference. Each wiring is one schedule under the law of Section 4.5, placed relative to Eq. (7), with ε the price of the shared support defined in Section 4.8. All three wirings are coupled. Every cell regenerates on current semantics at two or three grid points and one at-scale anchor, as deltas with their spread, paired with the kernel cells whose gains the cost buys.

Wiring	Independence given up, and its form	Touches (r, w)	Quality vs. reference
Sparse write, dense read	the read’s, outright (the parity wiring)	$(N, 1)$	reference
Shared-support routing	part of both: the shared projection	off, $O(1)$	[PLACEHOLDER: ε : shared-support vs. dense-read rows]
Level split	the write’s placement at one level	$(\sqrt{N}, \sqrt{N})$	[PLACEHOLDER: level-split vs. dense-read rows]

The three are configurations, and Table 5’s second column leaves open how to read a gap on each, which Appendix A.17 supplies. The dense-read column approximates nothing on either side, which is what makes it the reference.

Every cell of Table 5 runs the three at identical N and L against the dense-read reference, and reports each wiring’s quality where the grid places it.

7.3 Routing inspection

The grid reports accuracy, and Section 5 makes claims about the assignment behind it, so we read that assignment off the same runs. After a cell trains, its final validation pass is replayed on the exact checkpoint over the exact evaluation inputs and through the same code path, under instrumentation. Every quantity below therefore comes from the forward passes that produced the reported accuracy rather than from probe inputs the model never saw. Nothing trains again.

We read three quantities off those replays, in the vocabulary of Section 5. They are whether the realized coloring is proper, the mass on edges whose endpoints share a feature, and read/write agreement under π in the sense of Eq. (9).

[PLACEHOLDER: Per condition and N : the realized coloring’s properness, the mass on edges sharing a feature against that cell’s accuracy, and read/write agreement under π .]

8 Limitations

What this paper owes first is measurement. The theory grid of Section 7.1, the wiring surfaces of Section 7.2 and the kernel constants of Table 7 stand here as placeholders. The counts behind those constants belong to one machine and one toolchain, so they carry no publishable provenance and are owed again on the kernel generation a reader can download. The capacity-fair comparison with softmax attention is the one settled without a run, arithmetically at $N = L$ by Eq. (10), under the head shape and the exclusions stated with it.

Four limits are structural. RoLA folds identity into the address when it writes, so a read takes whatever its feature holds at full magnitude when another symbol put it there. Its normalized readout signals that substitution no more than a similarity readout signals its own, and what is ours is that the failure arrives abruptly rather than by degrees (Section 5.2).

A later read also cannot pull symbols apart once they share a feature. What keeps that pooling a form of generalization is the sizing argument and the sharing-safety condition of Section 5.3, properties of the contract a task imposes rather than guarantees the head supplies.

Kernel shape is the third, and it is the one difference from softmax attention we claim survives a partition coarser than the data: the limit belongs to the factorized map rather than to slot memory. Appendix A.3 states the sharpening rates and works out both attributions, and Appendix A.18 reads the same difference from the other two limits’ side.

Residency is the fourth limit. The paging of Section 6 holds resident bytes proportional to what a sequence wrote, so a map that writes densely inhabits the whole address space and pays for all of it. A limit on resident pages refuses the write outright, where a softer failure would degrade quietly.

Two directions stay out of scope, and this paper spends none of its evidence on either. A feature map trained under a per-token touch budget, with losses rewarding coherent supports, would move where the read cost sits. The intra-RoLA axes are the other: level configurations beyond the three of Section 7.2, read and write tying, and the map-style variants, which this paper defers.

9 Conclusion

Routing is a way of building a feature map, so a routed design is the recurrence it already ran with a new map in front of it (Proposition 1). Which recurrence to route is then the choice, and we took linear attention, the class softmax attention itself belongs to. The designs of the past year that make the same choice are instances of it at feature dimension N and not an alternative to it.

RoLA is the general form of a routed head on that class, and the head this paper builds. Its feature map is a factorized hierarchy of per-level entmax distributions, reaching $\prod_l b_l$ features from $\sum_l b_l$ router outputs. Its decay has one rate per level and advances only when a feature is written to, detached so that nothing reconstructs the state history in reverse. Its kernel bills the tiles a token touches, and meets attention’s arithmetic at $N = L$. The capacity theory turns a retrieval contract into a required N , and makes addressing entropy the quantity a comparison holds fixed.

The reduction also fixes what a proposal in this family owes. A routed architecture is a proposed feature map, and the class it belongs to is already characterized. Two things distinguish a member from its class: whether the map trains against matched baselines, and whether it is computable at a cost that justifies it.

The second means a realized kernel rather than a dense-mask simulation. This paper clears it at $N = 65,536$, and for the first it states the protocol that tests the map at controlled, reproducible addressing entropy, with the language-model scale that would settle it fully left to the sequel.

Acknowledgments

The author thanks Weigao Sun for early feedback on the idea and for his encouragement in pursuing it. Compute for this work was supported in part by a credit grant from CloudRift (<https://cloudrift.ai>).

References

- Arshia Afzal, Aviv Bick, Eric Xing, Volkan Cevher, and Albert Gu. Raven: High-recall sequence modeling with sparse memory routing. *arXiv preprint arXiv:2607.25357*, 2026. Concurrent work.
- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024a.
- Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *International Conference on Machine Learning (ICML)*, 2024b.
- Jimmy Ba, Geoffrey E Hinton, Volodymyr Mnih, Joel Z Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Loïc Cabannes, Pierre-Emmanuel Mazaré, Gergely Szilvasy, Matthijs Douze, Maria Lomeli, Ilze Amanda Auzina, Justin Carpentier, Gabriel Synnaeve, and Hervé Jégou. Sparse delta memory: Scaling the state of linear rnns through sparsity. *arXiv preprint arXiv:2607.07386*, 2026. Concurrent work.
- Beidi Chen, Tri Dao, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. Pixelated butterfly: Simple and efficient sparse training for neural network models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021. *arXiv:2009.14794*.
- Wanyun Cui. A hippocampus for linear attention: An exact memory for what the recurrent state forgets. *arXiv preprint arXiv:2607.02303*, 2026. Concurrent work.
- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.
- Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A Ortega. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations*, 2023.
- Mete Demircigil, Judith Heusel, Matthias Löwe, Sven Upgang, and Franck Vermet. On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 168(2):288–299, 2017. doi: 10.1007/s10955-017-1806-y.
- Jusen Du, Weigao Sun, Disen Lan, Jiaxi Hu, and Yu Cheng. MoM: Linear sequence modeling with mixture-of-memories. *arXiv preprint arXiv:2502.13685*, 2025.
- Ayoub Ghriss and Sourav Chakraborty. Kernelized linear attention: Breaking the capacity wall with symmetric cones. *arXiv preprint arXiv:2607.17419*, 2026. Concurrent work.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016. doi: 10.1038/nature20101.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Han Guo, Songlin Yang, Tarushii Goel, Eric P. Xing, et al. Log-linear attention. *arXiv preprint arXiv:2506.04761*, 2025a.
- Wentao Guo, Mayank Mishra, Xinle Cheng, Ion Stoica, et al. Sonicmoe: Accelerating moe with io and tile-aware optimizations. *arXiv preprint arXiv:2512.14080*, 2025b.
- Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated dltanet-2: Decoupling erase and write in linear attention. *arXiv preprint arXiv:2605.22791*, 2026. Concurrent work.
- Xu Owen He. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024.
- Harold S. Heaps. *Information Retrieval: Computational and Theoretical Aspects*. Academic Press, 1978.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

- Jerry Yao-Chieh Hu, Dennis Wu, and Han Liu. Provably optimal memory capacity for modern hopfield models: Transformer-compatible dense associative memories as spherical codes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2410.23126.
- Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying. In *International Conference on Machine Learning*, pages 21502–21521, 2024.
- William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984. doi: 10.1090/conm/026/737400.
- Pentti Kanerva. *Sparse Distributed Memory*. MIT Press, Cambridge, MA, 1988.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.
- Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- Dmitry Krotov. Hierarchical associative memory. *arXiv preprint arXiv:2107.06446*, 2021.
- Dmitry Krotov and John J. Hopfield. Dense associative memory for pattern recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1606.01164.
- Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Bingbin Liu, Jordan T Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Exposing attention glitches with flip-flop language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Christopher Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.
- Yuqi Pan, Yongqi An, Zheng Li, Yuhong Chou, Ruijie Zhu, Xiaohui Wang, Mingxuan Wang, Jinqiao Wang, and Guoqi Li. Scaling linear attention with sparse state expansion. *arXiv preprint arXiv:2507.16577*, 2025.
- Hao Peng, Jungo Kasai, Nikolaos Pappas, Dani Yogatama, Zhaofeng Wu, Lingpeng Kong, Roy Schwartz, and Noah A. Smith. ABC: Attention with bounded-memory control. In *Annual Meeting of the Association for Computational Linguistics*, 2022.
- Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- Michael Poli, Armin W Thomas, Eric Nguyen, Pragaash Ponnusamy, Björn Deiseroth, Kristian Kersting, Taiji Suzuki, Brian Hie, Stefano Ermon, Christopher Ré, Ce Zhang, and Stefano Massaroli. Mechanistic design and scaling of hybrid architectures. In *International Conference on Machine Learning*, 2024.
- Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, et al. HGRN2: Gated linear RNNs with state expansion. *arXiv preprint arXiv:2404.07904*, 2024.
- Jack W Rae, Jonathan J Hunt, Tim Harley, Ivo Danihelka, Andrew Senior, Greg Wayne, Alex Graves, and Timothy P Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Jack W Rae, Sergey Bartunov, and Timothy P Lillicrap. Meta-learning neural bloom filters. In *International Conference on Machine Learning*, 2019.
- Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, Victor Greiff, David Kreil, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. Hopfield networks is all you need. In *International Conference on Learning Representations (ICLR)*, 2021.
- Clayton Sanford, Daniel Hsu, and Matus Telgarsky. Representational strengths and limitations of transformers. In *Advances in Neural Information Processing Systems*, 2023.
- Clayton Sanford, Daniel Hsu, and Matus Telgarsky. Transformers, parallel computation, and logarithmic depth. In *International Conference on Machine Learning*, 2024.

- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Pingwei Sun, Yuxuan Hu, Jianchao Tan, Xue Wang, Jiaqi Zhang, Yifan Lu, Yerui Sun, Yuchen Xie, and Xunliang Cai. FG²-GDN: Enhancing long-context gated delta networks with doubly fine-grained control. *arXiv preprint arXiv:2604.19021*, 2026. Concurrent work.
- Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. Transformer dissection: An unified understanding for transformer’s attention via the lens of kernel. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: A unifying framework for designing sequence models with associative recall. *arXiv preprint arXiv:2501.12352*, 2025.
- David J. Willshaw, O. Peter Buneman, and H. Christopher Longuet-Higgins. Non-holographic associative memory. *Nature*, 222(5197):960–962, 1969. doi: 10.1038/222960a0.
- Kaicheng Xiao, Haotian Li, Liran Dong, and Guoliang Xing. RAM-net: Expressive linear attention with selectively addressable memory. *arXiv preprint arXiv:2602.11958*, 2026.
- Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, 2024.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024a.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning*, 2024b.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, 2024c.
- Chiyuan Zhang, Maithra Raghu, Jon Kleinberg, and Samy Bengio. Pointer value retrieval: A new benchmark for understanding the limits of neural network generalization. *arXiv preprint arXiv:2107.12580*, 2021.
- Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *International Conference on Learning Representations*, 2024a.
- Yu Zhang, Songlin Yang, Ruijie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, Peng Zhou, and Guohong Fu. Gated slot attention for efficient linear-time sequence modeling. In *Advances in Neural Information Processing Systems*, 2024b.
- Tianyu Zhao and Llion Jones. Fast-weight product key memory. *arXiv preprint arXiv:2601.00671*, 2026.

A Proofs

Notation follows Section 4 throughout, and each result appears in the form the body uses.

A.1 The reduction and its RoLA instance

Proof of Proposition 1. Slot separability makes the update N scalar recurrences over a common value, so slot s evolves without reference to any other slot. Unrolling it from $M_{0,s} = 0$ gives

$$M_{t,s} = \sum_{j \leq t} \left(\prod_{i=j+1}^t a_{i,s} \right) w_{j,s} v_j. \quad (12)$$

Input-affine writes make every coefficient in that sum a function of the input stream, so no term depends on the state it builds, and the sum is a fixed function of the stream.

With $a_{t,s} = 1$ the coefficient collapses and $M_{t,s} = \sum_{j \leq t} w_{j,s} v_j$, which read as an $N \times d_v$ matrix is $M_t = \sum_{j \leq t} \phi(k_j) v_j^\top$ under $\phi(k_j) = w_j$: the linear-attention state at feature dimension $d_{qk} = N$. The linear readout contracts it, $y_t = \sum_s r_{t,s} M_{t,s} = \phi(q_t)^\top M_t$ with $\phi(q_t) = r_t$. Nothing in the argument inspects what produced w and r , so a router and a projection are one object here.

For input-dependent $a_{t,s} \in (0, 1]$ put $A_{t,s} = \prod_{i \leq t} a_{i,s} > 0$ and rebase Eq. (12) on it, giving $M_{t,s} = A_{t,s} \sum_{j \leq t} (w_{j,s}/A_{j,s}) v_j$. The same contraction then holds under $\tilde{\phi}(k_j) = w_j \odot A_j$ and $\tilde{\phi}(q_t) = r_t \odot A_t$, elementwise, which is the gated form of linear attention with a per-feature decay on the map [Yang et al., 2024b]. The rebasing is a change of variables rather than an assumption, and positivity of a is what it needs. $\square$

Proof of Proposition 2. Eq. (1) puts leaves in bijection with mixed-radix tuples, and under that indexing the leaf factors of Eq. (3) are Kronecker products, $W_t = g_t^w \otimes_l p_{l,t}^w$ and $R_t = \otimes_l p_{l,t}^r$, since a leaf entry is the product of one entry per level. With decay disabled Eq. (4) meets the three hypotheses, leaves updating independently under coefficients from the feature map and the value projection, with Eq. (5) contracting linearly, so Proposition 1 gives linear attention at $d_{qk} = N = \prod_l b_l$ under $\phi(k_t) = W_t$ and $\phi(q_t) = R_t$, while the router emits $S = \sum_l b_l$ numbers per side and the kernel forms neither leaf vector. The mass channel of Eq. (5) is the same statement on an appended constant-one value coordinate, so the normalized readout is that identity in normalized form. With decay enabled the transition stays diagonal with coefficients determined by the input, so the rebasing above absorbs it into the feature map and the disabled case is no escape from the reduction. $\square$

A.2 Addressing

Lemma 1 (Radix codes and reachable supports). *For allocations p_l on their simplices, $\text{supp}(\otimes_l p_l) = \prod_l \text{supp}(p_l)$. Hence a point mass on leaf s^* is $\otimes_l \delta_{d_l(s^*)}$, so every injective assignment of instances to leaves is expressible and is a radix code on the digits; and an unreachable support needs at least two levels each spreading over at least two digits.*

Proof. The entry at leaf s is $\prod_l p_{l,d_l(s)}$, positive if and only if every factor is, which is the support claim. Taking $p_l = \delta_{d_l(s^*)}$ leaves one positive entry, at s^* . For the last part, suppose a target support is not the product of its per-level projections. Then it holds two leaves differing in at least two digits while omitting a digit-mix of them, so at least two levels project onto two or more digits; a target spreading at one level alone is a product and is reachable. $\square$

Lemma 2 (Injectivity and the layer-1 bound). *A state of N features loses nothing to the pigeonhole if and only if the map from sequence to written configuration is injective. Where the items a family can present at one position number at most N , an injective write addressing is available: distinct symbols then receive distinct leaves and a sequence’s address trace is unique. At layer 1 those items are the token vocabulary V , so $N \geq |V|$, one leaf per node of that layer’s graph, gives distinct tokens distinct addresses, independent of L and of the corpus. Injectivity of the configuration itself asks for more than the addressing, since the configuration is a sum of deposits.*

Proof. If two histories produce one configuration, every readout is a function of that configuration and returns one answer, so a contract separating them is unservable and no reader detects or repairs the loss; if the map is injective the configuration determines the history and a decoder exists. With at most N symbols available at a position, a writer giving distinct symbols distinct leaves is well defined, so the address trace of a sequence is unique, and at layer 1 the family is the token vocabulary by construction.

The trace is not the configuration. Under Eq. (4) with decay disabled the configuration is $M_{T,s} = \sum_{t \leq T} W_{t,s} v_t$, a set sum over the symbols written and blind to their order as attention is without positional features, so anagrams of one another write one configuration at every state size. Order enters through the write address, which the disjoint family of Lemma 3 supplies by construction, or through deposits that stop commuting, which is what the write-mass clock of Eq. (6) does. A leaf ages by the mass accumulated before each deposit, so prefixes that differ age the state differently, and the recurrence carries what positional features otherwise carry. The layer-1 bound is a statement about the addressing and not about the order. $\square$

The lemma states existence and settles neither whether the linear-per-level feature map with an entmax activation expresses an injective assignment nor whether training finds one, which is the gap Section 5.2 leaves to the grid. Sharing carries the further condition of Section 5.2, since a read addresses the feature rather than the items in it and no normalized readout, ours or a similarity one, reports the substitution.

Lemma 3 (Disjoint families are tight at $N = L$). *For disjoint per-position vocabularies $V_1, \dots, V_L$ of any sizes, under the contract that the addressing store every symbol of every sequence of the product family distinctly, the least N admitting a static feature map is L . A witness is expressible in RoLA’s form at S feature-map outputs.*

Proof. Necessity: one sequence presents L symbols at once and the contract forces them onto distinct features, so $N \geq L$. Sufficiency: color by position, sending every $u \in V_i$ to feature i . Two symbols share a feature only when they share a position, and disjointness means they never co-occur, so no sequence collides. Vocabulary sizes do not enter, positional encoding multiplying the domain rather than the number of colors.

For the witness, factor $L = \prod_l b_l$ and write the position in mixed radix, taking level l ’s write allocation to be $\delta_{d_l(i)}$; Lemma 1 makes that a reachable point mass. Standard positional encodings leave the position linearly decodable, so each level’s logits are a linear map of the input whose entmax returns the digit delta, and the coloring costs $S = \sum_l b_l$ feature-map outputs rather than N of them. Without systematic position structure the feature map has to memorize the same assignment at a cost proportional to the total symbol count. $\square$

Lemma 4 (Agreement, and what depth changes). *A feature map serves the contract sending query symbol u to stored symbol $\pi(u)$ when Eq. (9) holds, $\text{addr}^r(u) = \text{addr}^w(\pi(u))$ for every u , and otherwise the read marginalizes over the digits its cue leaves open.*

Proof. Under agreement the read concentrates on the leaf the write concentrated on. That leaf holds one value, so the mass fraction cancels in Eq. (5), $(Rm v)/(Rm) = v$, and any other leaf the read touches was never written and contributes zero to both sums (Section 4.3). The readout is then the stored value. If agreement fails at a digit group, the cue does not determine that group’s digit, so the read spreads over its branches and the active-leaf count grows with the number of instances matching the cue, which is the bill attention pays for a multi-match query. A write address may then use only features the retrieving query carries, per digit group. $\square$

A.3 Existence of a fixed map

A fixed map reuses one symbol-to-feature assignment everywhere, where attention computes an address per sequence from the data in front of it, so uniqueness across sequences looks unattainable even where single-sequence capacity is ample.

At layer 1 the family of Section 5.1 answers that objection, and Section 5.2 states the sizing that answer buys. What Lemma 2 adds is that two sequences landing one fact in one feature is the redundancy banking α prices. Position enters through the constructor and not through any repair, positional encoding yielding the disjoint family, where the coloring of Lemma 3 is tight and expressible in S parameters at the price of the L colors Section 5.2 charges that family. Eq. (9) governs the rest, deciding which contracts a fixed map can serve without asking where addresses come from.

Depth supplies the dynamism (Section 5.2), and what that leaves to settle is the map itself. The map at layer $k > 1$ is static as a function while its input is context-mixed, so the composed map from sequence to address is sequence-dependent and layer 1 is the only architecturally static layer. Attention’s data-dependent addressing was never a different mechanism, but the same fixed map applied to context-dependent inputs.

One difference from softmax attention survives this, and it is the one we claim: the shape of the kernel. Softmax sharpens exponentially in the gap between two scores, where a product of entmax allocations sharpens polynomially on each piece of its support. That belongs to the factorized map and not to slot memory, since one softmax over N features would sharpen exponentially at a producer of width N , which is the cost the factorization is there to avoid.

Pooling within a feature is not a second difference. Past what its own features separate, attention weights the keys it cannot tell apart alike and returns their blend, and its keys are fixed when written, so no later query separates them either. What differs is the profile of the failure, graded on a continuous key geometry against quantized on discrete leaves, and both shrink as the partition refines relative to the data and vanish at one token per feature, where RoLA expresses attention at attention’s price.

Normalization is not a difference either, the normalized regime of Section 3 being softmax’s own ground.

A.4 The shared-support error

Fix a shared-support level with logits z^r and z^w and write $p_*^r = \text{entmax}(z^r)$ and $p_*^w = \text{entmax}(z^w)$ for the allocations an independent level would solve. Eq. (8) instead solves once on the averaged logits, giving p with support U , and takes the read as $p\sigma$ and the write as $p(1 - \sigma)$ renormalized over U .

The tied case is an identity: $z^r = z^w = z$ gives $p = \text{entmax}(z)$ and $\sigma = 1/2$ at every branch, so the write is p after renormalizing and the read is $p/2$, whose uniform factor cancels in Eq. (5). Under the sum in place of the average the solve would receive $2z$, and $\text{entmax}(2z) \neq \text{entmax}(z)$, so the tied case would carry a temperature the ablation would confound with the mechanism.

Untied, the write error splits in two. Writing $p_*^w = p_*^w|_U + p_*^w|_{U^c}$, the second part is mass on branches outside the shared support, which the level loses rather than attenuates, and the first differs from the shared-support write because renormalizing over U redistributes what the truncation removed. Neither part shrinks as the logits sharpen. At order 2 on a two-branch level, $z^w = (1, -1)$ and $z^r = (-3, 3)$ give $p_*^w = (1, 0)$ and $p_*^r = (0, 1)$, while the averaged logits $(-1, 1)$ give $U = \{2\}$, so the shared-support write is the point mass $(0, 1)$, sitting on the branch its own logits ranked last.

The read side carries no renormalization, so a factor common to every branch is free under Eq. (5), and what survives is the truncation of $\text{supp}(p_*^r) \setminus U$ together with $p\sigma$ on U not being proportional to p_*^r there, so the head forms num_t and den_t over a support the read logits did not choose alone. Each side then realizes its intent projected through the shared support rather than that intent itself, which is the partial surrender of routing independence Section 4.5’s law charges the shared solve and the error ε prices.

The name is descriptive rather than literal, since the averaged solve keeps branches strong in either logit and so approximates the two supports taken together, while the witness above is where it departs. And because z^r enters the support solve, a point-mass write is not protected, which is how a shared support spends the write purity that the $N = L$ regime rests on (Section 5.3).

A.5 Level-split and shared-support geometries

Section 4.5 derives shared-support routing and a level split as two purchases of one guarantee, and the two are not two settings of one geometry. A shared-support level produces one support serving both sides, where a level split produces two supports sparse in different level coordinates, a write slab $\text{supp}(w_A) \times B$ against a read slab $A \times \text{supp}(r_B)$, and these do not coincide, so the geometries differ in kind rather than in degree. Table 5 prices them side by side.

The split generalizes further, since Section 4.5’s intersection guarantee needs one dense side per level and no more. The design space is then a per-level assignment of sides, in which a balanced choice puts both operations near $\sqrt{N}$ touches with coupling kept and each side still solving its own support, paying by splitting the exact-addressing exponent between read and write, and the released kernel runs such assignments as configuration. We leave that family unexamined, and as interior-regime exploration alone: a point-mass write needs a point mass at every level, so the natural sparse-write, dense-read configuration is the only member that writes injectively at $N = L$, and the guarantee of Section 6.1 for that regime belongs to it.

A.6 The decay sensitivity

Write $A_{t,s} = \text{keep}_{t,s} = (1 - \text{rate}_s)^{c_{t,s}}$ with $c_{t,s} = \text{sg}[w_{t,s}^{\text{leaf}}] \in [0, 1]$ the detached clock of Eq. (6). Because c carries no gradient, Eq. (4) stays affine in the state with input-determined coefficients, so the value and key-feature gradients travel through the deposit and the read alone and never ask for M_{t-1} .

The dial gradient does ask for it, and one state-sized plane supplies it. Define $P_{t,s} = \partial M_{t,s} / \partial \text{rate}_s$, which obeys

$$P_{t,s} = A_{t,s} P_{t-1,s} - c_{t,s} (1 - \text{rate}_s)^{c_{t,s}-1} M_{t-1,s}, \quad P_{0,s} = 0, \quad (13)$$

a forward recurrence of the state’s own shape, carried in the same scan. The backward pass then contracts output and final-state adjoints against P and needs no chunk-boundary snapshot of the state. The leaf rate is the product $\text{rate}_s = \prod_l \delta_{l,d_l(s)}$ over the dials on its digits, so $\partial / \partial \text{rate}_s$ scatters to the per-level dials by prefix and suffix products that stay defined at a zero dial, at S dials rather than N of them. Were the clock attached, c would depend on the key features and Eq. (13) would couple to it, which is the snapshot the stop-gradient removes.

A kernel rebasing a tile by a per-leaf scale, as one holding the exponent range in a usable band does, keeps that exact. The joint tile map on (M, P) is homogeneous of degree one in such a scale, so scaling both blocks on entry and unscaling both on exit changes neither, and the sensitivity survives the rebase, provided the scale is independent of the rate. A tile’s rate dependence enters through Eq. (13)’s source term and nowhere else, so differentiating the scale instead adds a multiple of the whole state, an error the size of the state rather than of the rounding.

A.7 Spread assignments and the nonnegative rank

Section 5.2 sizes for point-write assignment, each item entering one feature. Assignments that spread an item over several features are bounded by a different quantity, the nonnegative rank of the contract matrix rather than a coloring, the contract matrix being the one whose rows are the queries and whose columns the items, carrying the weight each query’s target puts on each item. Against the contracts this paper instruments that bound concedes nothing. A point lookup returns one item per query, so the matrix has a distinct row per query and full nonnegative rank on the items live

at once, and its bound degenerates to the one point writes already give. The relaxation is left to a contract class asking only for fixed mixtures, at ratios frozen at write time, so the reader scales features and never the members inside them, and what such a class demands in general is beyond this paper.

A.8 When sharing a feature is safe

Sharing one feature between two symbols is safe when no retrieval map in the contract addresses one of them without the other. A disjoint family makes this automatic, a query for position i intending only what position i stored. A shared family does not, and the failure is silent for both, neither readout verifying itself. Both are normalized averages, so a read returns a confident blend however poor the match, and past the demand a width provisions both weight wrong content confidently. What differs is where identity is kept. Attention keeps the key as data, one per occurrence, so discrimination degrades by degrees under packing pressure, and exponential capacity per unit of width holds that regime cheaply out of reach. A grid folds identity into the address at write and discards the residual, so a read intending u in a sequence holding only v takes v at full magnitude with nothing to grade it. The asymmetry is graded against quantized failure, a difference of regime and not of verification.

A.9 The per-pair candidacy term

Section 6.1 keeps the term that stays linear in N whatever a wiring touches. Enumerating the realized support instead would add a second kernel structure and the arbitration between the two, which at a bit-level constant costs more than the term does. The caveat is structural, and we state it rather than settle it: the term scales with NL where its removal mechanism does not, so the decision rests on today’s constants, and a size or a machine that moves them retires the term instead.

A.10 Constructing the presence-recall instrument

The contract of Section 7.1 is dense over what a sequence writes: every item position is a read, so coverage is total by construction and evaluation is exhaustive over those positions. That keeps a measured threshold sharp against the demand rather than smeared by sampling luck. The label class is planted per position, the generator choosing whether the target has occurred and then drawing an item that realizes the choice. The chance line is then one half at every supervised position, and no positional prior leaks into the score. A positive is planted with an asserted separation between the occurrence of $\pi(x_t)$ and the position that asks for it, so no read is served out of the residual stream.

Cells are small, since the axis is addressing rather than scale and cheap cells buy breadth. Table 6 is the configuration, and one of its settings carries a reason. The learning rate is swept per column from $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$, the best cell reported and the winner re-run at further seeds. A rate at which a column fails to train is then a point in the sweep rather than a verdict on the column, and no threshold rests on a single rate.

Table 6: **Training configuration for every cell of the grid.** Held identical across the four routed columns and the attention reference cells, and the winning rate’s cell per column and N re-runs at further seeds for its error bar.

Setting	Value
Backbone	two layers at model width 128
Training	40 epochs on 20,000 examples
Learning rate	swept per column from $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$, best reported, winner re-run at further seeds
Ladder in N	$\{16, 32, 64\}$ at demand 16; $\{32, 64, 128\}$ at demand 64
Attention’s key width	$\{4, 8, 16, 32\}$ at demand 16; $\{8, 16, 32\}$ at demands 32 and 64
Sequence length, vocabulary, batch, seeds	64 tokens at demand 16 and 256 at demand 64; vocabulary 8,192; batch 64; seeds 0 and 1 at the first stage

A.11 The wiring family: per-candidate development

Disjoint supports are then permanent dead directions with no re-entry signal, the gap the top- k lineage patches by recruiting with injected noise [Shazeer et al., 2017]. Attention sits inside this law, at one corner of it. It writes a point mass per occurrence and reads densely, which is the sparse-write, dense-read wiring with the write map supplied free by position in place of learned. Its own collapse at small key width is the supply-side failure of discrimination the theory charges every member.

Correctness survives. Untied sparse supports are exact already, the unwritten leaves of Section 4.3 being inert in both sums, and what the intersection buys is gradient on the leaves a task uses, through training over many steps. The

cheapest wiring there is falls to the law all the same. A point-mass write against an independently solved sparse read can promise overlap on no step, and it is untrainable by construction.

A guarantee holding for every pair of logits means at least one side’s realized support departs from an independent point mass: whenever the two intents disagree, some realized support departs from the singleton its own logits chose. Table 3 is what that leaves, the wiring family being the solution set of the guarantee inside one mechanism, classified by what each purchase pays with. The subsections below construct the three that pay. The default configuration takes the first, staying the sparse-write, dense-read topology of Section 4.1 under leaf mass decay, and Section 7 runs the three against it.

What it pays is touches: the write is one leaf and the read is all N of them. A query therefore touches the full width of every level and the only sparsity a kernel can harvest is on the write side. Nothing is approximated on either side, which is what makes this the reference the other two are read against (Section 7).

The payment lands on the write: a level that reads sparsely must write densely if the two supports are to keep meeting. The write therefore becomes a contiguous block of features rather than a point mass and leaks mass into its neighbors. Appendix A states the geometry formally, together with the wider family of per-level side assignments this paper leaves unexamined.

This is the deliberate exception at the agreement end, both sides sparse at $O(1)$ touches and $rw \ll N$. Its two supports are identical within a token, one solve producing them, so what exactness asks of it is across time: that the one map send a cue where it sent the content. Where the query carries the key itself that holds by construction, the same map on the same input, and the guarantee is as strong as the covering wirings’ at $O(1)$ touches.

What it is exposed to is a cue diverging from the content, where the burden is canonicalizing the two through that one map (Section 5). The covering wirings pay touches to make no demand on the cue, and the shared support pays no touches and makes that demand instead. Eq. (7) is why nothing reaches $O(1)$ on both sides while leaving each side its own support.

The coupling costs an approximation, from two sources decomposed in Appendix A. The first is that the shared solve drops a branch one side wanted. The shared support in general differs from the support of $\text{entmax}(z^r)$, so one side’s strongly negative logit can push the other side’s branch below the averaged threshold, losing its mass outright. The second is reweighting among the survivors. Renormalizing the write side redistributes the dropped mass, and $p\sigma$ is not $\text{entmax}(z^r)$ restricted to the shared support, so the head forms num_t and den_t over a support the read logits did not choose alone.

We write ε for the resulting task-level error: what a shared support costs. It measures how often a task reads outside where it writes, or the reverse, strongly enough for one support to get it wrong. That depends on the task’s read and write geometry, so we measure ε (Section 7).

A shared support and factorization charge different sides. Factorization constrains reads and leaves writes alone, since an injective assignment of instances to features is always expressible (Section 5). A shared support charges the write side: z^r enters the support solve, so a point-mass write is no longer the write logits’ own. At $N = L$, one feature per token with L the sequence length, write purity is the whole guarantee. It survives a shared support wherever the query carries the key itself, one solve taking cue and content to one leaf, and what a shared support spends is the case where the two diverge.

Each side carries one mass and no more, since per-level masses collapse into a side-wide product and add parameters without adding expressivity. A token shares content the same way, writing one value vector everywhere it writes. Neither factor is ever formed over leaves, the kernel folding the S per-level allocations instead (Section 6).

Each level carries one of two configurations. An independent level owns separate read and write allocations, each from its own solve, so a token may write to a branch that queries avoid. A shared-support level owns one support solve for both sides (Section 4.8). Level order sets the factorization and the layout, nothing semantic. The default configuration is sparse writes on the coarse levels above a leaf-most independent level with dense reads. Figure 2 draws a head whose levels put the sparsity on different sides.

Exactness then classifies what survives that cut. A level serves a read without error under either of two conditions. Under agreement both sides pin the level’s digit, at one touch a side, and the cue must carry that digit (Section 5). Under coverage one side spreads over all b_t digits and asks nothing of the cue there, at b_t touches on that side.

A dense read spreads over every branch of every level while the write stays a point mass, so the read support contains any write support whatever the two producers do. A side is dense because its level carries softmax, which cannot emit a zero, so the full support is a property of the activation and not of the logits. Every level is covered, and the read makes no demand on the cue at any of them.

Because a leaf factor is a product over levels, the two sides can take their sparsity at different levels, each keeping its own logits, solve and support. Every level is then dense on one side and sparse on the other, so no level is pinned by both and the supports meet through the product structure alone. Coverage still holds at every level, and the envelope at equal widths is $(\sqrt{N}, \sqrt{N})$, one square root a side.

A.12 The write-mass clock: lineage and detachment

The clock ignores that gain by design. Eq. (5) reads a ratio, so relative eclipse, a large later write overshadowing a small earlier one, is already answered for at read time. A clock charging decay in proportion to a share of the accumulated mass would depend on the state it ages and cross the input-affine boundary of Section 3. The per-token simplex allocation is the normalized clock unit that stays inside it.

In the gated linear-attention lineage the clock advances once per timestep, so a feature ages even when it takes no write [Yang et al., 2024b, Gu and Dao, 2023, Yang et al., 2024a, Qin et al., 2024]. Here the exponent is the leaf’s own write mass, so a feature ages in proportion to the interference it has taken on. That is the law we want when Section 5 sizes the state so that almost every feature is idle at almost every step. A zero in any stored level factor gives $w_{t,s}^{\text{leaf}} = 0$ and $\text{keep}_{t,s} = 1$, not approximately and not learned near one, so an unwritten feature keeps every bit.

Kimi Delta Attention gates per channel [Kimi Team et al., 2025], FG²-GDN makes the delta update’s learning rate per channel [Sun et al., 2026], and Gated DeltaNet-2 splits the erase gate from the write gate, arguing that the two should be decoupled [Hatamizadeh et al., 2026]. Tying them is the contrary design, and inside that family we know of no other gate that is an algebraic function of the feature’s own write mass. That is what carries the identity above into a backward pass with no state history. A state of 10^5 to 10^6 features needs that to be trainable and not only definable at that size, the largest trained N being Table 2’s open cell.

In the gated linear-attention lineage the clock advances once per timestep, so a feature ages even when it takes no write [Yang et al., 2024b, Gu and Dao, 2023, Yang et al., 2024a, Qin et al., 2024]. Here the exponent is the leaf’s own write mass, so an unwritten feature keeps every bit, which is the law we want once Section 5 sizes the state so that almost every feature is idle at almost every step (Appendix A.12).

The clock also runs detached. Data-dependent gating normally costs a backward pass its independence from the state trajectory. The stop-gradient leaves the recurrence affine in the state, so the backward pass needs no state history and no chunk-boundary snapshots (Appendix A). We scope the claim to the chunked gated-linear-attention family, where the gate advances at every timestep through a function parameterized apart from the write allocation. The finest-grained recent forms sharpen that separation.

A.13 Capacity and addressing: what factorization costs

That gap is what depth buys, and we pay for it in what the map can express. The feature vector is a product of per-level marginals, one simplex choice per level rather than a free point in $\mathbb{R}^N$. Mass on the leaves $(0, 0)$ and $(1, 1)$ alone is therefore unreachable, the product forcing all four cells. The state is freer: a sum of product-form deposits need not itself be product-form, so the memory is full rank while the addressing factorizes.

Factorization costs nothing where writes want point masses. The obstruction needs mass on two digits at two levels, so it never reaches a point mass, a delta on a leaf being the product of the per-level digit deltas. Any injective assignment of instances to features is a radix code (Appendix A). It charges spread read patterns instead, the side where a task’s reads leave room.

Attention provisions $\exp(\Theta(d_{qk}))$ near-orthogonal directions, the classical count a width admits [Johnson and Lindenstrauss, 1984], and materializes $O(L)$ of them per sequence as its cache; RoLA provisions N combinatorially and commits the features written into. Both are logarithmic-class addressing for an exponential space. The exponential count on the attention side is the modern Hopfield line’s capacity result read as a provisioning statement [Krotov and Hopfield, 2016, Demircil et al., 2017, Ramsauer et al., 2021, Hu et al., 2024], how many addresses a width can resolve rather than how many a retrieval requires.

Read that way they sit beside the demand account below. A combinatorial address space is the classical sparse-coding one [Willshaw et al., 1969, Kanerva, 1988], where a hierarchy runs over energy levels [Krotov, 2021]. Recall capacity has an existing treatment in the worst case: communication complexity bounds the embedding dimension an attention layer needs [Sanford et al., 2023], and a state of fixed size bounds what a recurrent model can copy or recall [Jelassi et al., 2024, Arora et al., 2024a]. The graph of Section 5.2 parameterizes that demand by distribution and contract.

Under the normalized readout of Eq. (5) that solver has two error-free cases. A read confined to features holding one value each is exact however far the write spread, the mass fraction cancelling as $(Rmv)/(Rm) = v$. A read spreading over features the sequence never wrote is exact as well, those features being inert (Section 4.3).

One failure mode remains, shared by every combination of spread and point addressing. Read support lands on features holding other values, at write-set ratios the read weights cannot cancel. Point writes with point reads expose the least of that surface and spread with spread the most. The choice among them is a dial between fidelity and robustness.

A.14 Regimes, licensing and the adversarial contract

Every feature then holds at most one live value, which is the first exact case above: the mass fraction cancels and the read returns what was stored. The least N that serves the contract is the chromatic number of the graph, the fewest colors a proper coloring takes (Figure 3). That is, the head needs one feature for every item a read has to keep apart from the others at the same time. It is not $\sum_i |V_i|$, and not $|V|^L$, which counts sequences that one coloring covers together.

Every quantity this paper calls a demand is that number evaluated on a particular graph, and sequence length, vocabulary sizes and window widths enter as parameters of the construction that produced the graph. The graph and its chromatic number are the whole of the demand, and that number is the most any contract over the realized sequences can ask for, so provisioning to it leaves every fulfillment pattern available. Restricted contract classes ask for less and never for more, which is what makes one number serve them all.

The quantity is also classical and computable with certificates on the families the instrument builds, so Section 7.1 stamps a solved demand on a cell before it trains. A demand theory no one can solve is a demand theory no one can test. What is left to an architecture is which colorings its map expresses, at what production cost, and whether training finds one, the subject of the rest of this section and of the grid of Section 7.1.

The first regime is the one random draws over a shared vocabulary present, multi-query associative recall among them, and the instrument of Section 7.1 runs in it, for that guarantee tier. The second is the coloring the factorization expresses structurally, at S parameters with levels as position digits (Appendix A), which is what makes the positional regime cheap. The third is induced by the contract, and its width is fixed before any architecture is chosen.

A read that never demands a superseded item apart from a live one leaves an interval graph of that width whatever the head does. Decay is the supply that meets that demand and not its source, since without it a superseded item keeps its mass in the feature and contaminates the normalized read, so the recency contract fails at N equal to the window.

A contract bills two resources, and features are only the first. Coloring the graph is a storage statement: $N = L$ colors it unconditionally, one feature per occurrence, and no count of features says which occurrence a read returns. The second resource is discrimination, and it sits in the map, since returning the intended occurrence asks the map producing the addresses to separate the symbols that can land in a feature. The graph fixes the colors a contract needs, so what a node count far above N pressures is the map that has to express the coloring over those nodes, and every design grows that side.

Softmax attention grows d_{qk} at $\Theta(\log |V|)$ in the node count, its similarity multiplying evidence across dimensions. A factorized map instead grows the sum $S = DN^{1/D}$ that it emits. Figure 1’s two panels are that law stated as a bill. Attention buys the cheaper address producer and pays in storage that grows with the context, where the factorization pays a polynomially costlier producer to hold its storage flat. Neither leaves that regime by provisioning features alone.

The number a contract requires is thus a functional of the distribution and the contract rather than a property of the head, and what changes across families is how one comes to know it. A disjoint family, which positional encoding constructs, asks for L , and a designer knows that in advance. A shared family, which token identity constructs, asks for the chromatic number of the realized graph, which that graph’s node count bounds from above, and which only measurement returns. Entmax triage quotients that graph to its costly edges, and α is set to match what survives, again a measurement. This paper measures neither.

None of this asks the map to be computed per sequence. At layer 1 the family is the token vocabulary, so N sized to that vocabulary gives distinct tokens distinct addresses, at a size that follows the vocabulary alone (Appendix A). Depth then supplies the dynamism, later layers seeing a family whose symbols carry sequence structure, so the demand falls from the graph of the whole distribution to the graph of one sequence (Appendix A.3).

How N must scale with L is a property of the sequences a task realizes rather than of L alone, the graph to color being the union over what a distribution presents. Natural language realizes a vocabulary $V(L)$ that grows as a sublinear power of the length of a context [Heaps, 1978], so its nodes, and with them its chromatic number, sit far below the adversarial completion.

Sizing N to that realized node count leaves a $D = 2$ factorization addressing it at width $O(\sqrt{|V(L)|})$ per token, which is $O(L^{1/4})$ at the exponent of one half that growth is commonly fitted at. The required N as a functional of the sequence distribution is the general sizing question, which this paper leaves open.

Provisioning below the adversarial demand is a bet that the graph a design faces is smaller than the adversarial one, and three things shrink it. Positions carrying the same item collapse onto one node, so L positions present fewer than L items. Edges carry a cost, the price of confusing the pair they join, and items whose confusion costs nothing may share a color at benefit, so only the costly edges bind. And only the edges some sequence realizes are there at all, which is the ladder above: a natural distribution realizes far fewer of them than the adversarial completion holds.

That ladder is literal at the first layer, whose items are tokens as they arrive. Deeper layers address learned representations, whose ambient space compounds combinatorially with depth, and the graph they color is manufactured by the mixing through the last two of those three. Canonicalization merges co-referent forms onto one node or leaves the edges between them costless, so the effective vocabulary shrinks toward referents.

The representations a layer realizes concentrate far below the combinatorial completion, which keeps the union graph small deeper in the stack while its ambient space grows. How far that concentration goes stays an observation. Layer 1 is the rung where neither reduction is available, which is why sizing is literal there and why the adversarial instrument is honest at it. The map is a fixed function, so the only lever mixing holds is the distribution it hands that map, and the law above applies unchanged to the image of the mixing.

The asymmetry between its two learned maps is $W_Q \neq W_K$ rather than one side learned against one side fixed. Its retrieval errors are attention-weight errors, the mirror of ours, and spread writes buy nothing at this size. A comparison here isolates the two read maps and the price of learning a write map attention gets free, and this size is what the kernel must survive (Section 6). Appendix A.3 delineates the one difference from attention that survives outside this regime.

That regime is an entry fee, pricing a disjoint family in which every symbol is distinct and independently addressable. Real sequences are redundant, so provisioning $N = \alpha L$ sizes the state to the information in a context rather than to its token count. Conflicts below that size are purposeful: quality tracks the mismatch between what the blend merges and what the contract separates rather than the number of conflicts. Near-duplicate content pooling its mass under the normalized readout is generalization wherever the contract does not separate what pooled.

What a head must address is which distinguishable symbols can occur at each position, the family $V_1, \dots, V_L$. Every statement below quantifies over such a family, and an origin is a constructor. Token identity gives the shared family, all $V_i = V$; positional encoding makes the per-position vocabularies disjoint; later layers grow the family with symbols carrying sequence structure. A query addresses with whatever cue it carries, so the disjoint and shared families are the positional and the content-associative cases. Eq. (9) states the demand that follows: a write must address using only what the retrieving query will carry.

Sizing starts from injectivity, which means that no two histories may end up as the same memory. The state loses nothing to the pigeonhole if and only if the map from sequence to written configuration is one-to-one. Where that fails, two histories produce one memory that no reader can detect or repair. An injective addressing is available whenever the items a family can present at one position number at most N , a coarse form of the demand the graph below states in full. Distinguishing orders asks for position in the address on top of it (Appendix A).

Sharing one feature between two symbols is safe when no retrieval map in the contract addresses one of them without the other. A disjoint family makes that automatic and a shared family does not. Past the demand both heads weight wrong content confidently, and the difference is that attention keeps identity as data and degrades by degrees where a grid folds identity into the address and fails abruptly (Appendix A.8). Two symbols that never co-occur may then share a feature for storage but not for retrieval, unless the cue the query addresses with covers everything that can land there.

How N must scale with L is a property of the sequences a task realizes rather than of L alone, the graph to color being the union over what a distribution presents. Natural language realizes a vocabulary growing as a sublinear power of context length [Heaps, 1978], so its chromatic number sits far below the adversarial completion (Appendix A.14).

That ladder is literal at layer 1, whose items are tokens as they arrive, and it is the rung where neither reduction is available, which is why sizing is literal there and why the adversarial instrument is honest at it (Appendix A.14).

That regime is an entry fee, pricing a disjoint family in which every symbol is distinct and independently addressable. Real sequences are redundant, so provisioning $N = \alpha L$ sizes the state to the information in a context rather than to its token count, and conflicts below that size are purposeful (Appendix A.14).

The asymmetry between its two learned maps is $W_Q \neq W_K$ rather than one side learned against one side fixed, so a comparison here isolates the two read maps and the price of learning a write map attention gets free (Appendix A.14).

Every quantity this paper calls a demand is that number evaluated on a particular graph. The graph and its chromatic number are the whole of the demand, and that number is the most any contract over the realized sequences can ask for, so provisioning to it leaves every fulfillment pattern available (Appendix A.14).

The quantity is also classical and computable with certificates on the families the instrument builds, so Section 7.1 stamps a solved demand on a cell before it trains. What is left to an architecture is which colorings its map expresses, at what production cost, and whether training finds one.

One failure mode remains, shared by every combination of spread and point addressing: read support landing on features that hold other values, at write-set ratios the read weights cannot cancel. The choice among them is a dial between fidelity and robustness (Appendix A.13).

A.15 The reduction’s ladder, and the family table read row by row

The identity is undecayed linear attention. Diagonal transitions are what elementwise decay and gating produce, and they commute, so an unrolled product of them collapses to a per-token elementwise reweighting the keys absorb. The attention form survives, and this class is the reduction’s domain. A rank-one modification of the identity fails to commute. The delta rule’s $A_t = I - \beta_t k_t k_t^\top$, at write strength β_t on key k_t [Schlag et al., 2021], leaves the effective key of the value v_t a history-dependent matrix, so the write-time-fixed key the form factors each stored pair through is absent from that unrolling.

Gating modifies the past state too, and stays inside the form because its transitions commute. The boundary is commutativity, not state modification. The addressing may itself be routed, as in the sparse distributed line; what the folding acts on is the recurrence. That recurrence remains linear-scannable, and the designs on that side read as fast weights in their own papers. Dense A_t is the general linear RNN. The reduction is a boundary inside that ladder.

MoM is the exhibit: its framework’s general update is plain-additive and reducible, while the instantiation it evaluates employs Gated DeltaNet memories. The same router lands on either side according to the rule dropped into it, Proposition 1’s condition classifying recurrences and not routers.

Raven, SSE and RoLA write sharply while reading densely, and they pay differently for it. Raven reads by a softmax over all slots against a per-slot key state, so cost and state both carry the slot count [Afzal et al., 2026], and SSE reads every partition by default [Pan et al., 2025]. RoLA’s normalized readout is linear in N with no per-slot key. A support capped at k on both sides cannot express those read contracts at fixed k , since a query with more than one match needs a dense marginalization over the slots holding them.

The recurrence off the diagonal gives up the chunked parallel form. An affine, slot-separable recurrence folds a chunk with one matrix product and needs no state history in the backward pass, where a transition off the diagonal fails to commute across the chunk and brings WY-form representations and intra-chunk solves with it [Yang et al., 2024c,a]. A state-dependent deposit needs the state at every write point besides. That parallel-form cost is one of the reasons Section 1 gives for taking the affine side. Whether the affine side is itself the limitation is the hypothesis this paper tests.

Three things separate ours from the rest of the family. The first is scale. Our feature map is exact entmax over a factorization sized for N between 10^5 and 10^6 , a runtime count of memory entries the sequence fills, where product-key expert banks reached 10^6 units as weights [Lample et al., 2019, He, 2024]. That is three orders above the 10^2 to 10^3 at which the routed alternatives report a trainability fix, collapse-prevention noise or a proxy gradient (Sections 4.5 and 7.1). This paper’s kernel addresses $N = 65,536$ (Section 6), and Table 2’s open cell is the largest trained one.

The second is the decay. Advancing by received write mass is prior art we carve out below; ours are two properties of that clock, a rate learned per level and exact detachment in place of a proxy gradient. The tying follows: the gate is not a second free function of the input but the write allocation itself, one map serving as the addressing and as the clock (Section 4.4). Concurrent work applies the same principle to cache admission [Cui, 2026].

What we add is a hierarchy in place of a flat factorization, so levels carry coarse and fine writes at once, each under its own configuration. Exact α -entmax replaces softmax with top- K , which removes the occasion for two consequences RAM-Net reports of that choice: the high-variance gradients it reports when one softmax addresses all slots directly, and a sensitivity to the degree of the factorization. Its clock is one global exponent whose derivative at the decay boundary it trains through a proxy gradient, where ours is a learned rate per level, detached in place of smoothed. That is what yields a backward pass with no state history (Section 4.4). Both clocks are diagonal, so both heads stay inside the attention form of Proposition 1. The capacity theory and cost model of Sections 5 and 6 go unreported there.

Separability makes the update N independent scalar recurrences, each unrolling to $M_{t,s} = \sum_{j \leq t} (\prod_{i > j} a_{i,s}) w_{j,s} v_j$. Affine writes make every coefficient there a function of the inputs alone, so the sum is a feature map evaluated on the stream. The linear readout then contracts it against r_t , which is $\phi(q_t)^\top M_t$. The hypotheses omit positivity, so the

proposition also covers the gated baselines, which are customarily unnormalized. The paper works throughout in the normalized, nonnegative regime, where every routing vector lies on a simplex. The generalization to arbitrary feature maps follows, and we leave it aside here.

The normalized regime is a fact about RoLA and not a simplification of it. Write mass, the clock that reads it and entmax as a projection onto the simplex all need a magnitude only a nonnegative coefficient supplies (Section 4), and softmax attention’s weights are a distribution over slots on the same ground. Classical normalized linear attention fell out of use over the instability of its running denominator [Katharopoulos et al., 2020], and the readout of Section 4.3 is that same ratio. Keeping it is what puts the head on softmax attention’s ground, and it scopes the paper.

The rows engineer ϕ for address structure, asking how many resolvable slots a map indexes and at what router cost, where the older line on the same design surface engineers it for similarity fidelity instead [Choromanski et al., 2021, Arora et al., 2024b, Zhang et al., 2024a]. Those goals compose, and we leave approximation quality to that line.

The third is a kernel at $N = L$ (Section 6). Without one, describing a slot memory as linear attention at $d_{qk} = 10^6$ restates the design and does not price it. SSE is the member that already reads its own update as linear attention, and it realizes the sparsity by masking a dense state or by reordering tokens by partition, reporting no utilization figure [Pan et al., 2025]. The reduction it states carries no cost result with it.

What identifies the class is that folding a chunk’s transitions into its keys asks them to commute across tokens, which means the order they are applied in cannot matter, and which the delta rule’s transitions fail to do [Schlag et al., 2021] (Section 3). It is efficient to implement in chunked form, and it scales to the slot counts this paper addresses. And what we are trying to match is softmax attention, which the class contains, so the delta rule’s extra machinery buys nothing here. The capacity theory says how large a state has to be before the class reaches that recall (Section 5). What separates the family is then which map, at which addressing cost (Figure 1).

The released layer takes the levels, their widths, each level’s activation and side assignment, and the clock as configuration. A reducible design whose activation the layer supplies in exact form is then a setting of it, RAM-Net’s per-part product softmax among them. Truncating an address to its strongest entries is not such an activation, so the members that do it, Raven and SSE, sit inside the reduction and outside that configuration space. The delta update sits outside both. Section 7 states what the account predicts and the protocol that tests it.

Capacity is then the product of the widths where the router’s output is their sum, $\prod_l b_l$ slots from $\sum_l b_l$ numbers, which is the mixed-radix arithmetic the design runs on. Its slots also age on a clock of received write mass, the exponent of the decay being the write weight a slot takes in. A slot that takes no write keeps every bit, which is what a model running without a fixed horizon needs.

That reading covers the whole family: routed keys under a delta rule give a routed fast-weight memory, and under an affine recurrence they give the designs studied here. Several members place themselves inside linear attention already, but each stating it for the one router it builds, and SSE states the identification for its own update, as a row-sparse formulation for linear attention [Pan et al., 2025].

Read as mechanisms they look like anything but feature maps: N is large, the routing vector is sparse, and nobody writes it down as one. The recurrence is indifferent to what produced the vector. A routed slot memory is linear attention under a particular feature map, and the proposition names the map.

Much of the line since has varied the update rule, and that axis reads as a ledger of what a write off the diagonal costs to parallelize. Gating multiplies the state by an input-dependent retention before each deposit, a diagonal step that keeps a chunk foldable with one matrix product [Yang et al., 2024b, Gu and Dao, 2023, Qin et al., 2024], where the delta rule moves the transition off the diagonal and test-time training off a closed affine form altogether, each paying for it in a costlier parallel form [Schlag et al., 2021, Yang et al., 2024c, Wang et al., 2025]. That is the other recurrence class, and routed versions of it exist; the choice of Section 1 puts them outside this paper’s scope, and Section 3 states where the two classes part.

The second means a realized kernel rather than a dense-mask simulation, and a proposal carrying neither leaves its contribution undetermined. Theory about the class stands outside this, capacity results holding on their own. This paper clears the second at $N = 65,536$, and for the first it states the protocol that tests the map at controlled, reproducible addressing entropy, with the language-model scale that would settle it fully left to the sequel.

A.16 Cost: the quantization, the ledger’s riders and the residue

The same measurement carries the other half, the candidate (owner, tile) pairs that hold nothing live for their owner and skip every step. A cost model prices the quantum, not the mean.

Two consequences follow, and both invert the intuition that a sparse memory should touch as little as it can. The first is that support size is nearly free. A support larger than needed costs little once it has paid for its tiles. No cost argument then remains for a hard per-token budget. Product-key memories adopt top- k for the compute it saves [Lample et al., 2019], where Raven adopts it to limit interference [Afzal et al., 2026], and only the first motive is the one this arithmetic answers. Section 4.2 can spend support freely and let the activation set its own boundary, which makes exact entmax affordable.

What that gives is structural: an address whose contiguity a flat address of the same capacity would have to learn. Whether the router, the module that emits the per-level distributions, then places related tokens under one coarse digit depends on what it learns, which we leave unmeasured here. Hierarchy is a coherence prior in that sense, the cost-side counterpart of the addressing argument of Section 5.

In effect, a head may provision far more features than the memory it ever occupies. Translation costs one table indirection per granule against the $16d_v$ state entries that granule holds, a part in a thousand at $d_v = 64$. The measured figure is. What decode walks and what a checkpoint serializes follow the same support, and the released kernel carries both.

We price the comparison with softmax attention per pair and then count pairs, since attention’s per-token cost grows through the sequence and any one token’s price, the final token’s above all, misstates the total. Attention pays $d_{qk} + d_v$ multiply-accumulates per query-key pair: one dot product for the score, one to accumulate the weighted value. That is $2d_v$ at the standard head shape $d_{qk} = d_v$, which the ledger below assumes; at $d_{qk} < d_v$ it pays less and the ledger under-claims. RoLA pays about d_v per (token, leaf) pair, the value contraction alone, addressing having moved into the factorized router at D multiplies per token.

Decode returns the same answer, and a per-step advantage is easy to claim by accident. At step t attention pays $2td_v$ against a cache of size t , while RoLA pays Nd_v at every step, because we size the state for the end of the sequence from the start. Attention is therefore cheaper below $t = N/2$ and dearer above. Over L steps the totals are L^2d_v against LNd_v . Decode is the triangle paid one row at a time, and no phase tilts the ledger.

A bit of bookkeeping runs against a multiply-accumulate at a ratio near 10^{-4} , which we have not measured and state only to an order of magnitude. The residue therefore stays invisible until a wiring touches fewer than $N/10^4$ features a token, so the slope-zero line of Figure 5(b) is predicted to flatten onto a floor of that order rather than onto zero.

The quantization cuts both ways, and no average shows it. A head addressing $N = 65,536$ features leaves a candidate block of 64 holding fewer live features than the quantum it is billed for, since the k -step count cannot fall below one wherever anything is live.

The second consequence is that clustering, not sparsity, is the quantity worth optimizing. Write ρ for the fraction of features a token touches. A tile is live if any of its B_T tokens touches it, so the kernel pays for the tokens’ combined footprint. That density is near $\min(1, B_T\rho)$ when the tokens touch unrelated features and near ρ when they agree. At $\rho = 1/1024$ and $B_T = 32$ that is a factor of 32 against a factor of 1024.

Provisioned capacity and resident memory come apart, which is the same argument on the storage side. The state is paged in software at a fixed granule of 16 leaves, and a page is admitted from the realized write support the producer has already computed. Resident bytes therefore scale with what a sequence wrote rather than with the provisioned count. The feature count is an address space, and a model pays for the fraction of it that it inhabits, which under sparse writes is the small quantity.

That size is the adversarial regime of Section 5.3, sparse writes with dense reads, where the write router buys nothing the contract needs. Eq. (10) therefore says we reach attention’s arithmetic while paying to learn what attention gets free. The wiring that meets the equation is the sparse-write, dense-read one. Making both sides dense adds a second fold over the state and the intra-tile corrections that go with it, a bounded constant over attention and the price of a memory every token may rewrite rather than append one row to.

What we add is where a routed head lands on it. Think of the state as a grid cut into blocks. A work item pairs an owner, that is, one block of B_C leaf features, with a tile of B_T tokens. The device stages or skips it whole, at a cost of $\lceil K/16 \rceil$ k -steps, that is, one matrix-multiply step per 16 of the K features live in it.

Nearby tokens choosing the same features is coherence, that is, tokens in one tile agreeing on where they write, and the factorized address supplies part of it by layout. The physical order of the features follows digit order. A coarse digit therefore selects a contiguous region, and every fine decision beneath it lands in tiles that region has already paid for.

The affine recurrence makes one prediction about memory. Its backward pass reconstructs no history of the state, which Appendix A proves from the detached clock, so what a step must retain per token stays flat in L . A pass that checkpoints

the state at every chunk boundary retains a state per boundary and pays that growth. Training memory per token is predicted flat in L , not measured.

A.17 The presence-recall instrument: construction and columns

The cue and the content are different tokens by construction, so a head that answers has to store x where a later token pointing at x will look. Eq. (9) is then a property training has to find rather than one the cue supplies. Every position that carries an item carries a label, except the first of a sequence, whose prefix is empty and whose answer is constant. The two classes are planted in balance at each labeled position, so chance is one half and a threshold in N is sharp against it.

The answer being one bit is what leaves the feature count alone in the measurement. A multi-query lookup returns a value of $\log_2 |V|$ bits, so its accuracy mixes addressing with retrieval and a partly resolved address still returns part of the answer, where presence recall asks only whether the target is there. The negatives supply separation from the other side. A position whose target has not occurred must answer no.

A feature holding two items that a sequence realizes together then turns a present item into a false positive for the absent one, so the contract forces every such pair apart on the realized graph. The demand is that graph’s chromatic number, a complete graph over the items a sequence draws puts the number at the item count, and the collapse threshold keeps the form Section 5.2 gives it.

The demand-16 ladder runs $\{16, 32, 64\}$, from the demand itself upward, since the implementation floors a routed head at 16 features, so the crossing is measured at the higher demand and bounded at the lower. Either way the provisioning ratio α of Section 5.2 runs through one against a demand that was measured. Softmax attention is the only other party, and it never appears on that axis, its state being a cache that grows with the context.

Every cell here is a trained model, its router and per-level clock optimized end to end and not fixed by construction. We run no head-to-head against the routed family of Table 2, because there is no size both sides run at. The routed alternatives on the diagonal report N between 10^2 and 10^3 , against the 10^5 to 10^6 this architecture targets. The two methods that do sit in our range, Sparse Delta Memory at 2.3×10^5 [Cabannes et al., 2026] and FwPKM at 2.6×10^5 [Zhao and Jones, 2026], are built on the other recurrence, the delta rule for one and a state-dependent gradient step for the other.

We enter attention twice. It runs first at a key width ample for every demand, which establishes that the cells are solvable at all. A routed failure at some N is then a statement about capacity, where an attention score far below saturation would say the construction is measuring difficulty and not demand. That calibration is the instrument’s floor, and no routed column can supply it.

It runs second as a second validation of the account, on the axis its own capacity law lives on. A head of key width d_{qk} packs $\exp(c d_{qk})$ addresses at a packing constant c set by the interference one tolerates (Section 5), a reading concurrent work reaches through spherical codes above a Welch interference floor [Ghriss and Chakraborty, 2026]. Its collapse threshold therefore moves with the logarithm of the demand where a routed one moves with the demand itself.

Sweeping d_{qk} over the same solved graphs makes c a measurement of ours, and we label c fitted wherever it appears. Three solved demands enter it, at 16, 32 and 64, the middle one running attention alone since the routed law is read off the outer pair, so the slope is a fit through three points.

Section 4.5’s law places the three: each is one schedule for a structurally guaranteed read-write intersection, and the second column of Table 5 is what each gives up to hold it. Untied sparsity, two independently solved sparse supports and the wiring that law excludes, gives up nothing and forfeits the guarantee. The three span the axis between coverage and agreement, at the envelopes Table 3 lists, and running them on one set of graphs is how the grid spans it. The ablation measures which payment is cheapest on task quality, not which mechanism is better.

A trained model enters the same figure as a marker. We read its measured s , γ and a off the trained router and place it on the surface, which locates that cell on the knob axes and prices nothing: a cost quoted for a cell is timed on the cell. What the sweep yields are its fitted slopes and the figures, properties of the sweep rather than of any point in it.

The same generator carries contracts and graph families this paper leaves unrun, and each extends the instrument in one direction. Clustered, overlapping and reuse constructions move the demand against the item vocabulary, against the count a sequence writes and against a natural sharing distribution, where the presence contract holds the graph complete. A partial contract asks about part of the item set and moves the demand without changing one token of the distribution.

A conjunctive contract asks whether every target of a token precedes it rather than whether one does, still answering in one bit. The demand grows toward the union of the targets, since a shared feature cannot tell one present target from

two. Those are the sequel’s instrument, and this paper’s confirmation runs on the one contract that isolates the feature count.

The three are configurations, and Table 5’s second column leaves open how to read a gap on each. The dense-read column approximates nothing on either side, which is what makes it the reference. A gap on the level split reads as a spread write, through the block write of Section 4.7, rather than as a support one side never got. A gap on the shared support is the error ε . That column’s write purity at $N = L$ holds where the cue carries the content itself and is at stake where the two diverge, which is the divergence π puts under test in Section 7.1.

A query re-presents the key its pair was stored under, so the read cue is the write content, and a head passes with one symmetric addressing map, never learning a correspondence between the two sides. That is also the regime in which shared-support routing holds Eq. (9) structurally (Section 4.5), so the wiring whose exactness most needs testing is the one the task exempts.

A grid that sweeps a quantity it cannot compute is sweeping a knob. Every segment therefore solves the demand of the graph it actually realized, a sample realizing a subgraph of what the construction describes. On the complete graphs this paper runs, the solver’s lower and upper bounds meet, so each cell sits at a demand that is solved and not assumed.

The contract is dense over what a sequence writes: every item position is a read, so evaluation is exhaustive and a measured threshold stays sharp against the demand. Appendix A.10 gives the planting that holds the chance line at one half and keeps any read from being served out of the residual stream.

We run two conditions, complete graphs at demand 16 and at demand 64, over sequences the generator draws. Sequences run 64 tokens at the lower demand and 256 at the higher. A sequence draws half its condition’s pool, 8 items of the 16 and 32 of the 64. The demand and the count a sequence writes therefore sit a factor of two apart in every cell. π runs over the pool of its condition, inside a model vocabulary of 8,192 tokens.

We run four routed columns over the feature count N on each: the three wirings of Section 4.5, and a dense control at one level, which is linear attention at N features with no factorization to express. The demand-64 ladder runs N over $\{32, 64, 128\}$ and brackets its solved demand from below and from above.

One more contrast runs at the lower demand, and it is about the clock rather than the feature count. The write-mass clock of Section 4.4 advances a feature in proportion to the mass it takes in, so a spread write ticks each of the features it touches a little. A clock that instead advanced every feature in the write’s support by a fixed step would tick them all fully. The two coincide on an exact point write, that is, wherever a token writes to one feature and no more, so the sparse-write column is a control cell and the clocks can separate only where writes spread. We pair the two clocks over the three wirings and the same ladder in N at demand 16, and read the decay-off column off the cells already described.

The second question is what read sparsification costs and what it buys. The paper’s sparse-regime claims rest on the answer: the kernel gains of Section 6 need a query to touch a small part of the state, and the default configuration reads densely. The three wirings of Section 4.5 are sparse writes with dense reads, a level split, one level carrying the write and another the read, and shared-support routing. They are structural variants of one head, run at identical N and L , so a difference between them is the wiring.

We sweep the wirings over three knobs, each a quantity the cost and the learning condition both depend on. The occupancy s is the fraction of the state a sequence writes at all, in other words how much of the memory a sequence ever touches. The coherence γ is the tile-level agreement among neighboring tokens’ supports that Section 6 charges for. The alignment a is the fraction of reads whose support meets the write that stored the target. Each wiring is a surface over those knobs.

The touch exponents are the falsifiable part, since they are slopes and not constants. The envelope of Section 4.5 is a ceiling on what one token may touch, where the exponent is how that ceiling grows in N , so adaptivity moves the constant a confident cue realizes and not the rate. A measured slope tests the wiring even where no token pays its envelope.

The grid reports accuracy, and Section 5 makes claims about the assignment behind it, so we read that assignment off the same runs. After a cell trains, its final validation pass is replayed on the exact checkpoint over the exact evaluation inputs and through the same code path, with instrumentation recording the per-level distributions as they are produced. Every quantity below therefore comes from the forward passes that produced the reported accuracy rather than from probe inputs the model never saw. Nothing trains again.

The account puts the loss on those edges rather than on the count of items a sequence writes. The third is agreement, whether the addresses the read and the write sides assign meet in the sense of Eq. (9), measured under π as the address a token reads at against the address its target wrote at.

We record three predictions here as well. The write-mass clock sits at its decay-off twin on both spreading wirings, and a column materially below that twin would say the clock erodes retention under spread writes. The fixed-step clock degrades with the width of the write support. It is worst on the level split, whose dense write level ticks that whole level at every step, and milder on shared-support routing, whose support spans two addresses. An ordering that does not track support width falsifies it. The control pair holds, and a split there is an instrument finding about the sparse-write column’s realized support and not a result about clocks. The fixed-step clock is an instrument for this contrast alone: it runs on the training oracle, which is the engine every cell of the grid trains on and the only one that implements it.

The negatives supply separation from the other side: a feature holding two items a sequence realizes together turns a present item into a false positive for the absent one, so the contract forces every such pair apart on the realized graph. The demand is that graph’s chromatic number, and the collapse threshold keeps the form Section 5.2 gives it.

It runs second as a second validation of the account, on the axis its own capacity law lives on. A head of key width d_{qk} packs $\exp(c d_{qk})$ addresses at a packing constant c set by the interference one tolerates (Section 5), so its collapse threshold moves with the logarithm of the demand where a routed one moves with the demand itself.

Section 5.2 leaves one route below the demand open, for contracts asking only for fixed mixtures of stored items, and a presence query falls outside that class, by the nonnegative rank argument stated there. What these cells measure is the chromatic number itself.

We read three quantities, in the vocabulary of Section 5. The first is whether the realized coloring is proper, that is, whether items live at once take distinct features, which is the assignment the exact case rests on. The second is the mass sitting on edges whose endpoints share a feature, read against the accuracy of the cell it came from.

Two questions follow from the theory, and this section answers both: whether the capacity account predicts where retrieval fails, and what each wiring costs a token. The grid that answers the first is also this paper’s evidence that the head trains, and Section 7.3 reads the assignment behind that grid off the same runs. Every number reported here is produced by the head of Section 4 on the semantics stated there. Where a measurement is still missing a placeholder names it, and the repository’s MANIFEST.md lists the run that fills each one.

The first question is whether the capacity account predicts where retrieval fails. Section 5.2 makes that account a number. What a fixed-state head pays for is the demand, the chromatic number of the co-occurrence graph a distribution realizes, and not the count of items a sequence writes. Multi-query associative recall leaves that account untested [Arora et al., 2024a], and the contract is what stops it.

A.18 The kernel-shape difference, read from both sides

Kernel shape is the third, and it is the one difference from softmax attention we claim survives a partition coarser than the data. The two concentrate differently on the same evidence, and the limit belongs to the factorized map rather than to slot memory. Appendix A.3 states the sharpening rates and works out both attributions. The abrupt failure of the first limit is that difference read from the other side, a continuous key geometry against discrete leaves, and the pooling of the second is that same difference again.

B Limits of the cost evidence

The planner selects gathering live columns before the matrix-multiply on routed-sparse cells and rejects it on dense ones. The sign reversal is the finding rather than a caveat on it. On the at-scale cell of Section 6, gathering retires instructions with decay off and more with decay on, and paired interleaved timing puts the gathered kernel below its dense counterpart. On a saturated cell of the same topology the same mechanism adds instructions instead, so gathering serves routed sparsity and is not an unconditional optimization. [PLACEHOLDER: Instructions retired by gathering at the at-scale cell with decay off and with decay on, the paired interleaved timing of the gathered kernel against the dense one, and the instruction cost of the same mechanism on the saturated cell; rental-class hardware, ratified protocol.]

Table 7: **The cost quanta and what isolates each**, per candidate (owner, tile) pair. Config: $N = 65,536$ as two levels of width 256, $d_v = 64$, 8 heads, $B_T = 16$, $B_C = 64$ except where an isolating pair varies it. The middle column is structural, and every coefficient is isolated by the cell or cell pair in the last column, which varies one extent, rather than fitted. [TODO: Restore the work column, in warp instructions, from hardware instruction counters on rental-class hardware under the ratified protocol, decay as the range across which the selections are re-run.]

Term	Scales with	Isolated by
Gather, fixed part	candidate tile	the B_C pair
Gather, quantum part	$\lceil B_C/16 \rceil$	the B_C pair
Readout, fold, panels	k-step	the at-scale cell
Intra-level contraction	k-step	the write- $k=1$ pair
Leaf mass decay	k-step	the B_C pair

What binds the cost model is the configuration choices it reproduces, its measured (B_T , gather) selections, rather than its absolute predictions, which we record rather than assert. One per-tile term decides part of that agreement while the kernel source refutes the work it names. Its attribution is therefore un-identified, and we do not re-fit its value to compensate. Removing it leaves every row under-predicted, the one-sided shape of a model pricing only the terms it measured, and decay is likewise overpriced on the one at-scale cell where the extents move. We record that direction rather than correcting it, since setting a second decay constant would spend the one cell that could then validate it. [PLACEHOLDER: Selections reproduced of selections measured, how many of them the un-identified per-tile term decides, the one-sided residual once it is removed, and decay’s residual factor at the at-scale cell; same protocol.]